

RAPPORT DE PROJET TUTORÉ

RIKI

Application mobile de suivi de musculation
Framework React Native & base de données SQLite

Étudiant	Ilias AIT
N° étudiant	82508880
Formation	L3 Informatique — IED Paris 8
Année universitaire	2025-2026
Date de remise	Mars 2026

Table des matières

1. Introduction et contexte

- 1.1. Contexte et problématique
- 1.2. Objectifs et contributions
- 1.3. Structure du document

2. État de l'art

- 2.1. Le marché des applications de fitness
- 2.2. Analyse des solutions existantes
- 2.3. Technologies de développement mobile
- 2.4. Bases de données embarquées
- 2.5. Gestion d'état dans React Native
- 2.6. Positionnement du projet

3. Analyse et conception

- 3.1. Analyse des besoins
- 3.2. Cas d'utilisation
- 3.3. Conception architecturale
- 3.4. Modélisation de la base de données
- 3.5. Conception de l'interface utilisateur

4. Implémentation

- 4.1. Environnement technique
- 4.2. Couche Modèle — Entités métier
- 4.3. Couche Données — DatabaseHelper
- 4.4. Couche Métier — Services
- 4.5. Couche Présentation — React Contexts
- 4.6. Couche Interface — Écrans
- 4.7. Thème et design system

5. Validation et évaluation

- 5.1. Stratégie de test
- 5.2. Tests unitaires — Code et résultats
- 5.3. Tests de widgets
- 5.4. Tests de performance
- 5.5. Synthèse des résultats

6. Discussion

- 6.1. Analyse critique
- 6.2. Difficultés rencontrées
- 6.3. Compromis effectués

7. Conclusion et perspectives

8. Annexes

1. Introduction et contexte

1.1 Contexte et problématique

1.1.1 Contexte scientifique et industriel

Le marché mondial des applications de santé et de fitness connaît une croissance soutenue depuis le début des années 2010, accélérée de manière spectaculaire par la pandémie de COVID-19 en 2020. Les confinements successifs ont poussé des millions de personnes à chercher des solutions numériques pour maintenir une activité physique régulière en dehors des salles de sport. Selon les études de marché, le secteur des applications de fitness pèse plusieurs milliards de dollars et continue de croître à un rythme annuel de 15 à 20 pourcent.

Dans ce paysage, la musculation (ou entraînement en résistance) occupe une place particulière. Contrairement aux activités cardio-vasculaires où les métriques sont relativement simples (distance, durée, fréquence cardiaque), la musculation exige un suivi précis de multiples variables : la charge utilisée (en kilogrammes), le nombre de répétitions effectuées, le nombre de séries, le temps de repos entre les séries, et l'organisation des exercices en programmes structurés. L'accumulation de ces données permet de calculer des indicateurs de progression comme le volume d'entraînement (somme des charges x répétitions), le tonnage hebdomadaire, ou les records personnels (PR) sur chaque exercice.

La littérature scientifique en sciences du sport confirme que le suivi méthodique de l'entraînement est un facteur clé de progression. Le principe de surcharge progressive — augmenter graduellement le stress imposé au muscle — ne peut être appliqué efficacement que si le pratiquant dispose d'un historique précis de ses performances passées. Sans outil numérique, cette tâche de suivi est traditionnellement reposée sur un carnet papier, une méthode laborieuse et sujette aux erreurs qui ne permet pas de visualiser facilement les tendances à long terme.

1.1.2 Identification du problème

L'analyse du marché des applications de suivi de musculation révèle un paradoxe frappant. D'un côté, les applications leaders (Strong, Hevy, JEFIT) proposent des fonctionnalités complètes et une interface soignée, mais imposent un modèle économique freemium avec des abonnements mensuels allant de 5 à 15 euros. La version gratuite est volontairement bridée (limite sur le nombre de programmes, statistiques avancées payantes, export de données payant) pour inciter à la souscription. De l'autre côté, les alternatives gratuites souffrent soit d'une interface obsolète (FitNotes, non mis à jour depuis 2023), soit d'un manque de fonctionnalités critiques (pas de graphiques de progression, pas de gestion de programmes structurés, pas de détection automatique des records).

Un deuxième problème, moins visible mais tout aussi important, concerne la confidentialité des données. La quasi-totalité des applications du marché imposent la création d'un compte utilisateur et stockent les données d'entraînement sur des serveurs distants (cloud). Or, ces données — fréquence d'entraînement, charges soulevées, évolution des performances — constituent des données personnelles liées à la santé physique, soumises au RGPD en Europe. L'utilisateur n'a souvent aucune visibilité sur la manière dont ces données sont exploitées (revente à des tiers,

ciblage publicitaire, profilage).

Troisièmement, la surcharge fonctionnelle (feature bloat) est un problème récurrent. Beaucoup d'applications tentent de tout faire : suivi nutritionnel, intégration de montres connectées, dimension sociale avec partage de séances, programmes coaching payants, marketplace de programmes. Cette surcharge nuit à l'expérience utilisateur en rendant l'interface confuse et la navigation lente, alors que le besoin fondamental reste simple : enregistrer rapidement ses séries entre deux exercices en salle de sport.

1.1.3 Questions de recherche

Ce projet s'articule autour de trois questions de recherche complémentaires :

- **Q1** : Comment concevoir une application de suivi de musculation qui soit à la fois complète (programmes, séances, statistiques) et minimaliste dans son interface, sans sacrifier l'une pour l'autre ?
- **Q2** : Quelles sont les performances atteignables avec un stockage exclusivement local (SQLite embarqué) pour les requêtes agrégées (statistiques, graphiques) après des mois d'utilisation intensive et l'accumulation de milliers d'entrées ?
- **Q3** : Le framework React Native permet-il d'atteindre une qualité d'interface et une fluidité comparables aux applications natives (Swift/Kotlin) pour ce type d'application orientée saisie de données et visualisation ?

1.2 Objectifs et contributions

1.2.1 Objectifs principaux

L'objectif premier de ce projet est de concevoir et développer une application mobile fonctionnelle répondant aux besoins identifiés. Concrètement :

- Développer une bibliothèque de 65 exercices prédéfinis couvrant 8 groupes musculaires, avec la possibilité pour l'utilisateur de créer ses propres exercices.
- Permettre la création de programmes d'entraînement structurés par jours, avec 3 modèles prédéfinis (Push/Pull/Legs, Upper/Lower, Full Body).
- Implémenter un écran de séance optimisé pour la saisie rapide en salle : sélection d'exercice, poids, répétitions, chronomètre de repos intégré, détection de records personnels en temps réel.
- Intégrer des visualisations graphiques de la progression : courbe d'évolution des charges par exercice, répartition du volume par groupe musculaire.
- Garantir un stockage 100% local sans compte utilisateur, respectant la confidentialité des données de santé.

1.2.2 Objectifs secondaires

Au-delà du produit final, ce projet vise à démontrer la maîtrise d'un cycle de développement complet, de l'analyse des besoins à la livraison d'un produit fonctionnel, en passant par la conception architecturale, l'implémentation, les tests et la documentation. Il constitue une mise en pratique concrète des enseignements de L3 en bases de données relationnelles (normalisation, index, requêtes agrégées SQL), en génie logiciel (design patterns, architecture en couches, tests unitaires) et en développement mobile (gestion d'état, cycle de vie des widgets, asynchronisme).

1.2.3 Hypothèses de travail

Nous posons trois hypothèses :

- **H1** : SQLite, bien que limité à un accès mono-utilisateur local, est suffisant pour le cas d'usage visé (un seul utilisateur par appareil, pas de synchronisation multi-appareils).
- **H2** : Le framework React Native permet d'atteindre un rendu à 60 images par seconde de manière constante grâce à son moteur de rendu Skia, même avec des listes longues et des graphiques animés.
- **H3** : Le pattern React Context (useReducer) offre un compromis optimal entre simplicité de mise en oeuvre et performance de l'application pour un projet de cette envergure (12 écrans, 5 entités, ~2500 lignes de code).

1.3 Structure du document

Ce rapport est organisé en huit chapitres. Le chapitre 1 (présent) introduit le contexte, les objectifs et les hypothèses. Le chapitre 2 présente un état de l'art détaillé du marché des applications fitness, des frameworks mobiles et des bases de données embarquées. Le chapitre 3 couvre l'analyse des besoins (fonctionnels et non-fonctionnels), les cas d'utilisation et la conception architecturale. Le chapitre 4, le plus volumineux, détaille l'implémentation avec des extraits de code commentés pour chaque couche de l'architecture. Le chapitre 5 présente la stratégie de test, le code des tests et les résultats obtenus. Le chapitre 6 propose une discussion critique des forces, faiblesses et compromis du projet. Le chapitre 7 conclut et ouvre des perspectives d'évolution. Enfin, le chapitre 8 regroupe les annexes (inventaire du code, manuel d'installation, guide utilisateur, références).

2. État de l'art

Ce chapitre dresse un panorama détaillé de l'écosystème dans lequel s'inscrit notre projet : le marché des applications fitness, les solutions concurrentes, les technologies de développement mobile, les bases de données embarquées, et les patterns de gestion d'état. Cette analyse justifie les choix techniques effectués pour RIKI.

2.1 Le marché des applications de fitness

Le secteur des applications de fitness constitue un segment dynamique de l'économie numérique. Depuis 2015, la catégorie « Health & Fitness » figure parmi les 10 catégories les plus téléchargées sur Google Play et l'App Store. La pandémie de 2020 a provoqué un pic de téléchargements sans précédent, les salles de sport étant fermées dans de nombreux pays pendant plusieurs mois.

On peut distinguer plusieurs catégories d'applications dans cet écosystème. Les applications de coaching guidé (Nike Training Club, Freeletics) proposent des programmes prédéfinis avec des vidéos démonstratives. Les applications de suivi d'activité généraliste (Strava, Google Fit, Apple Health) se concentrent sur les métriques agrégées (pas, calories) et les activités cardio. Enfin, les applications spécialisées en musculation (Strong, Hevy, JEFIT, FitNotes) se focalisent sur l'enregistrement des séries et la progression des charges. C'est dans cette dernière catégorie que RIKI se positionne.

Le modèle économique dominant est le freemium : l'application est gratuite au téléchargement mais les fonctionnalités avancées (statistiques détaillées, nombre illimité de programmes, export de données) nécessitent un abonnement mensuel ou annuel. Ce modèle est rentable pour les éditeurs mais frustrant pour les utilisateurs qui découvrent les limitations après avoir investi du temps à saisir leurs données. L'effet de verrouillage (lock-in) est d'autant plus fort que l'historique d'entraînement accumulé rend le coût de changement d'application élevé (perte de données).

2.2 Analyse des solutions existantes

2.2.1 Strong

Strong est considéré comme la référence du marché. L'application se distingue par une interface minimaliste et élégante, une saisie de données extrêmement fluide (le champ de poids se pré-remplit avec la dernière valeur utilisée), et des graphiques de progression clairs. La base de données d'exercices est complète avec des illustrations animées. Cependant, la version gratuite limite l'utilisateur à 3 programmes personnalisés. L'abonnement Pro coûte environ 5 euros par mois ou 30 euros par an. Les données sont synchronisées via iCloud/Google Drive, ce qui rend l'application dépendante d'un compte cloud.

2.2.2 Hevy

Hevy est un concurrent plus récent qui mise sur la dimension sociale : l'utilisateur peut partager ses séances, suivre les entraînements de ses amis, et s'inspirer des programmes de la communauté. L'interface est moderne et colorée, avec un thème sombre par défaut. Le modèle freemium est plus agressif que Strong : les graphiques de progression détaillés et les statistiques par groupe musculaire sont réservés aux abonnés Pro (environ 9 euros par mois). L'application nécessite obligatoirement un compte et une connexion internet pour fonctionner, ce qui la rend inutilisable en mode avion ou dans les sous-sols de certaines salles.

2.2.3 JEFIT

JEFIT est l'une des plus anciennes applications de musculation (lancée en 2010). Sa bibliothèque d'exercices est impressionnante (plus de 1 300 exercices avec des animations 3D), et elle propose un système de plans d'entraînement très détaillé. Cependant, l'interface accuse son âge : la navigation est complexe, les écrans sont surchargés d'informations, et le design n'a pas été modernisé pour suivre les standards Material Design 3. L'abonnement Elite coûte environ 7 euros par mois et débloquent les statistiques avancées et la suppression des publicités (la version gratuite affiche des bannières publicitaires intrusives pendant l'entraînement).

2.2.4 FitNotes

FitNotes mérite une mention spéciale car c'est l'une des rares applications gratuites, sans publicité et open-source (disponible sur GitHub). Son approche est minimaliste : pas de programmes, pas de modèles, juste un calendrier et la possibilité d'enregistrer des séries. Les graphiques sont basiques mais fonctionnels. Le problème majeur est l'abandon du projet : la dernière mise à jour date de 2023, et l'application n'est disponible que sur Android (pas de version iOS). L'interface utilise encore des composants Material Design 1 (ActionBar, FloatingActionButton d'ancienne génération).

2.2.5 Synthèse comparative

Critère	Strong	Hevy	JEFIT	FitNotes	RIKI
Prix	5€/mois	9€/mois	7€/mois	Gratuit	Gratuit
100% Offline	Non	Non	Partiel	Oui	Oui
Open source	Non	Non	Non	Oui	Oui
Graphiques	Oui	Payant	Payant	Basique	Oui
Programmes	3 max	Illimité	Illimité	Non	Illimité
Exercices prédéfinis	~200	~300	~1300	~0	65
Compte requis	Oui	Oui	Oui	Non	Non
Détection PR	Oui	Oui	Non	Non	Oui
Chronomètre repos	Oui	Oui	Oui	Non	Oui
Dernière MAJ	2025	2025	2024	2023	2026
Framework	Natif	React Native	Natif	Natif Android	React Native

Tableau 1 — Comparaison détaillée des applications de suivi de musculation

2.3 Technologies de développement mobile

2.3.1 Développement natif

Le développement natif consiste à écrire du code spécifique à chaque plateforme : Swift ou Objective-C pour iOS, Kotlin ou Java pour Android. Cette approche offre les meilleures performances possibles et un accès complet aux API du système (capteurs, notifications, widgets). Cependant, elle nécessite de maintenir deux bases de code distinctes, doublant le coût de développement et de maintenance. Pour un projet universitaire réalisé par un seul étudiant, cette approche est difficilement viable dans les délais impartis.

2.3.2 React Native (Meta)

React Native, créé par Facebook (Meta), permet de développer des applications mobiles en JavaScript/TypeScript en utilisant des composants natifs. L'architecture repose sur un bridge JavaScript qui communique avec les composants natifs de chaque plateforme via un protocole asynchrone. Cette architecture a été récemment refondée (New Architecture avec Fabric et TurboModules) pour réduire les problèmes de performance du bridge. React Native bénéficie de l'écosystème npm massif et de la popularité de React pour le web, mais les animations complexes et le rendu personnalisé restent moins fluides qu'en natif, car le rendu dépend des composants natifs de chaque plateforme.

2.3.3 React Native (Google)

React Native adopte une approche radicalement différente : plutôt que d'utiliser les composants natifs de chaque plateforme, React Native dessine chaque pixel de l'interface avec son propre moteur de rendu (Skia, remplacé progressivement par Impeller). Cette approche garantit un rendu pixel-perfect identique sur iOS et Android, élimine les problèmes de compatibilité entre versions d'OS, et permet des animations à 60 fps constants sans bridge.

Le langage JavaScript, utilisé par React Native, supporte la compilation AOT (Ahead-of-Time) pour les builds de production, produisant du code machine natif performant. En mode développement, la compilation JIT (Just-in-Time) et le hot reload permettent de voir les changements de code en moins d'une seconde, ce qui accélère considérablement les itérations sur l'interface. L'écosystème de packages (pub.dev) est riche et activement maintenu, avec des packages de qualité pour SQLite (react-native-sqlite-storage), les graphiques (react-native-svg), la gestion d'état (React Context, Riverpod, Bloc), et l'internationalisation (intl).

2.3.4 Justification du choix de React Native

Notre choix s'est porté sur React Native pour les raisons suivantes :

- **Code unique** : un seul code JavaScript pour iOS et Android, essentiel pour un développeur seul dans un délai contraint.
- **Performance native** : la compilation AOT et le moteur Skia garantissent 60 fps sans bridge, crucial pour une interface de saisie rapide.
- **Hot reload** : les itérations UI sont quasi instantanées, permettant de tester rapidement différents designs pour l'écran de séance.
- **Material Design 3 natif** : React Native intègre nativement les composants MD3, ce qui facilite la création d'une interface cohérente et moderne.
- **Écosystème** : react-native-sqlite-storage, react-native-svg et React Context sont des packages matures et activement maintenus.

Critère	Natif	React Native	React Native
Langage	Swift/Kotlin	JavaScript/TS	JavaScript
Rendu	Composants OS	Bridge + natif	Moteur propre (Skia)
Performance	Excellente	Bonne	Très bonne
Code partagé	0%	~85%	~95%
Hot reload	Non	Oui	Oui (sub-seconde)
Taille APK	Petite	Moyenne	Moyenne (~15 Mo)
Courbe apprentissage	Moyenne	Facile (si JS)	Moyenne

Tableau 2 — Comparaison des frameworks de développement mobile

2.4 Bases de données embarquées

2.4.1 SQLite

SQLite est le moteur de base de données le plus déployé au monde : il est intégré dans chaque appareil Android et iOS, dans la plupart des navigateurs web, et dans d'innombrables applications embarquées. Sa conception « serverless » (pas de processus serveur séparé, la base est un fichier unique) le rend particulièrement adapté au stockage local sur mobile. SQLite supporte les transactions ACID, les requêtes SQL complètes (JOIN, GROUP BY, sous-requêtes, fonctions de fenêtrage), les index B-tree pour des recherches en $O(\log n)$, et les triggers. Sa limite principale est l'accès mono-écriture concurrent (un seul processus peut écrire à la fois), ce qui n'est pas un problème pour une application mobile mono-utilisateur.

2.4.2 Alternatives NoSQL : Hive et Isar

Hive est une base de données clé-valeur (NoSQL) conçue spécifiquement pour React Native. Ses avantages sont la vitesse d'écriture (pas de surcharge SQL), la simplicité de l'API (open/read/write sans mapping), et la compatibilité web (Hive fonctionne aussi dans le navigateur). Cependant, Hive ne supporte pas les requêtes relationnelles (JOIN, GROUP BY), ce qui rendrait les calculs statistiques (volume par groupe musculaire, évolution des charges) très coûteux à implémenter côté JavaScript (chargement de toutes les données en mémoire puis filtrage/agrégation manuelle).

Isar, successeur de Hive par le même auteur, résout partiellement ce problème en proposant un langage de requête plus riche avec des filtres composés et du tri. Néanmoins, Isar reste un projet relativement jeune (première release stable en 2022) avec une communauté plus restreinte que SQLite. Pour un projet académique, la maturité et la documentation exhaustive de SQLite constituent un avantage décisif.

2.4.3 Realm

Realm (désormais MongoDB Realm) est une base de données objet conçue pour le mobile. Ses performances en lecture/écriture sont excellentes, et il supporte la synchronisation cloud native (Atlas Device Sync). Cependant, Realm présente deux inconvénients pour notre projet : (1) la taille du SDK augmente significativement l'APK final (plusieurs Mo de bibliothèques natives), (2) le modèle de données objet (pas de tables relationnelles) s'intègre mal avec les concepts de bases de données vus en cours de L3, ce qui réduirait l'intérêt pédagogique du projet.

2.4.4 Justification du choix de SQLite

SQLite s'impose comme le choix optimal pour notre projet. Les requêtes agrégées SQL (SUM, MAX, GROUP BY, JOIN) sont essentielles pour les statistiques de l'application : calcul du volume par groupe musculaire, évolution des charges max par exercice, nombre de séances par semaine. Ces calculs seraient fastidieux et moins performants avec une base NoSQL. De plus, la modélisation relationnelle (exercices, programmes, séances, séries avec leurs clés étrangères) constitue une application directe des cours de bases de données de L3.

Critère	SQLite	Hive	Isar	Realm
Type	Relationnel	Clé-valeur	Document	Objet
SQL support	Complet	Non	Partiel	Non (propriétaire)
JOIN/GROUP BY	Oui	Non	Non	Non
Taille SDK	Intégré OS	~200 Ko	~1 Mo	~5 Mo
Maturité	20+ ans	~5 ans	~3 ans	~10 ans
Sync cloud	Non (local)	Non	Non	Oui (Atlas)

Tableau 3 — Comparaison des bases de données embarquées pour React Native

2.5 Gestion d'état dans React Native

La gestion d'état est le problème central de toute application React Native non triviale. React Native étant déclaratif (l'UI est une fonction de l'état : $UI = f(state)$), chaque changement d'état déclenche la reconstruction des widgets concernés. Le défi est de propager ces changements efficacement (sans reconstruire toute l'arborescence) et de manière maintenable (séparation logique/UI).

2.5.1 setState

La méthode la plus basique est `setState()`, intégrée à React Native. Elle déclenche la reconstruction du widget courant. Simple et adaptée pour l'état local (par exemple, l'état d'un champ de saisie), elle devient impraticable pour l'état partagé entre plusieurs écrans (liste des exercices, séance en cours, etc.) car elle ne fournit aucun mécanisme de propagation dans l'arbre de widgets.

2.5.2 React Context

React Context (par Remi Rousselet) est le package officiellement recommandé par l'équipe React Native. Il repose sur `InheritedWidget` (mécanisme natif de React Native pour la propagation de données dans l'arbre) enveloppé dans une API simple. Le pattern `useReducer` + `Consumer` permet de notifier les widgets intéressés lorsqu'un état change, sans référence explicite entre le React Context et les widgets. `MultiReact Context` permet d'injecter plusieurs providers dans l'arbre, chacun gérant un domaine fonctionnel.

2.5.3 Riverpod

Riverpod (aussi par Remi Rousselet) est la version 2.0 conceptuelle de React Context. Il corrige les limitations de React Context : dépendance à `BuildContext` (état inaccessible en dehors de l'arbre de widgets), impossibilité d'avoir deux providers du même type, et absence de compile-time safety. Cependant, Riverpod 2.x utilise la génération de code (annotations `@riverpod` + `build_runner`), ce qui ajoute une étape de build et une complexité non négligeable pour un projet de cette envergure.

2.5.4 Bloc

Bloc (Business Logic Component) impose une architecture stricte basée sur les streams JavaScript : les Events entrent, les States sortent, et le Bloc encapsule la transformation. Cette séparation rigoureuse est excellente pour les grandes applications avec des flux complexes, mais introduit beaucoup de boilerplate (classes Event, State, Bloc) pour chaque fonctionnalité. Pour notre application avec des flux CRUD simples, Bloc serait surdimensionné.

2.5.5 Justification du choix de React Context

React Context offre le meilleur rapport simplicité/puissance pour notre cas d'usage. Ses avantages concrets pour RIKI sont : (1) pas de génération de code (contrairement à Riverpod 2.x), (2) API intuitive (useReducer + Consumer), (3) séparation claire logique/UI, (4) recommandation officielle React Native. Le passage ultérieur à Riverpod resterait possible sans réécriture majeure car les deux partagent le même modèle mental.

2.6 Positionnement du projet

RIKI se positionne dans un créneau précis : celui d'une application de suivi de musculation gratuite, complète, offline, open-source, et respectueuse de la vie privée. Aucune application existante ne réunit toutes ces caractéristiques. FitNotes s'en approche mais manque de fonctionnalités (pas de programmes, pas de graphiques avancés, pas de détection de records) et n'est plus maintenue. Strong est l'application la plus aboutie fonctionnellement mais impose un modèle payant et un stockage cloud. Le projet répond aux trois lacunes identifiées : gratuité complète, indépendance vis-à-vis du cloud, et simplicité d'interface sans compromis sur les fonctionnalités essentielles.

3. Analyse et conception

3.1 Analyse des besoins

3.1.1 Besoins fonctionnels

Les besoins fonctionnels ont été identifiés par l'analyse croisée de trois sources : (1) les fonctionnalités des applications concurrentes, (2) les retours d'utilisateurs sur les stores (Play Store, App Store), et (3) l'expérience personnelle en tant que pratiquant de musculation depuis plusieurs années. Ils sont organisés en quatre modules fonctionnels :

Module	ID	Fonctionnalité	Priorité
Exercices	F01	Bibliothèque de 65 exercices classés par groupe musculaire	Haute
Exercices	F02	Fiche détaillée par exercice (description, consignes)	Haute
Exercices	F03	Recherche textuelle sur nom et groupe	Haute
Exercices	F04	Filtre par groupe musculaire (FilterChip)	Haute
Exercices	F05	Création d'exercices personnalisés	Haute
Exercices	F06	Modification et suppression d'exercices perso	Moyenne
Programmes	F07	Création de programmes structurés par jours	Haute
Programmes	F08	3 modèles prédéfinis (PPL, Upper/Lower, Full Body)	Moyenne
Programmes	F09	Attribution d'exercices avec séries/ reps cibles	Haute
Programmes	F10	Démarrer une séance depuis un programme	Haute
Séances	F11	Saisie rapide poids/répétitions (clavier numérique)	Haute
Séances	F12	Chronomètre de repos avec presets (60s, 90s, 120s)	Moyenne
Séances	F13	Détection automatique des records personnels	Haute
Séances	F14	Historique de toutes les séances passées	Haute
Séances	F15	Détail par séance (séries, volume, durée)	Haute
Stats	F16	Tableau de bord synthétique (séances/semaine, volume)	Haute
Stats	F17	Graphique évolution charges max par exercice	Haute
Stats	F18	Graphique volume par groupe musculaire (30 jours)	Moyenne

Tableau 4 — Besoins fonctionnels détaillés

3.1.2 Besoins non-fonctionnels

Les besoins non-fonctionnels définissent les contraintes de qualité du système :

-
- **Performance** : temps de réponse inférieur à 100ms pour toute interaction utilisateur. Les requêtes agrégées SQLite doivent rester sous les 50ms même avec 10 000 séries en base.
 - **Compatibilité** : Android 8.0+ (API 26), couvrant plus de 95% du parc Android actuel. Le support iOS est architecturalement prévu mais non testé (faute de matériel Apple).
 - **Maintenabilité** : architecture en couches permettant de modifier une couche sans impacter les autres. Code commenté et structuré.
 - **Fiabilité** : les données saisies pendant une séance ne doivent pas être perdues en cas de fermeture accidentelle (persistance au moment de la validation).
 - **Ergonomie** : utilisable d'une seule main en salle de sport, avec des zones de tap suffisamment grandes (minimum 48x48 dp selon les guidelines Material Design).

3.2 Cas d'utilisation

Les cas d'utilisation principaux couvrent les scénarios typiques d'un pratiquant de musculation utilisant l'application au quotidien.

3.2.1 CU1 : Démarrer et enregistrer une séance

L'utilisateur arrive en salle, ouvre l'application. Depuis le dashboard, il appuie sur « Démarrer une séance » (ou choisit un programme puis « Démarrer »). L'écran de séance active s'ouvre. Il sélectionne un exercice dans le dropdown, entre le poids et les répétitions, appuie sur +. La série s'ajoute à la liste. Entre chaque série, il appuie sur un bouton preset (60s, 90s, 120s) pour lancer le chronomètre de repos. Si la charge dépasse le record personnel, un badge « PR » apparaît en temps réel. En fin de séance, il appuie sur « Terminer », confirme, et la séance est sauvegardée en base.

3.2.2 CU2 : Consulter sa progression

L'utilisateur veut savoir s'il progresse au développé couché. Il navigue vers l'onglet Stats, sélectionne « Développé couché barre » dans le dropdown. Un graphique LineChart s'affiche avec l'évolution de sa charge max sur chaque séance. Il peut toucher un point pour voir la valeur exacte. Sur le même écran, un graphique BarChart montre la répartition du volume par groupe musculaire sur les 30 derniers jours, lui permettant d'identifier un éventuel déséquilibre (par exemple, trop de pectoraux, pas assez de dos).

3.2.3 CU3 : Créer un exercice personnalisé

L'utilisateur pratique un exercice qui n'est pas dans la bibliothèque (par exemple, un exercice spécifique à sa salle). Il navigue vers l'onglet Exercices, appuie sur l'icône + en haut à droite. Le formulaire de création s'ouvre : il saisit le nom, sélectionne le groupe musculaire dans un dropdown, et optionnellement ajoute une description. L'exercice apparaît dans la bibliothèque avec un badge « personnalisé ».

3.3 Conception architecturale

L'architecture de RIKI repose sur une séparation en trois couches, chacune avec une responsabilité clairement définie. Ce découpage respecte le principe de séparation des préoccupations (Separation of Concerns) et facilite les tests unitaires (chaque couche est testable isolément).

Couche	Responsabilité	Classes principales
Données (Data)	Persistence SQLite, CRUD	DatabaseHelper, modèles (toMap/fromMap)
Métier (Business)	Règles de calcul, logique	StatistiquesService, TimerService
Présentation (UI)	Affichage, interaction	React Contexts (useReducer), Screens, Widgets

Tableau 5 — Architecture en couches

3.4 Modélisation de la base de données

3.4.1 Schéma relationnel

Le schéma comprend 5 tables normalisées en troisième forme normale (3NF). Chaque attribut non-clé dépend uniquement de la clé primaire, éliminant les dépendances transitives et les anomalies de mise à jour.

Table exercices

Champs : id (PK), nom, groupe_musculaire, description, est_personnalise

Catalogue des exercices. Le champ est_personnalise distingue les exercices créés par l'utilisateur des exercices prédéfinis, permettant de protéger ces derniers contre la suppression accidentelle.

Table programmes

Champs : id (PK), nom, description, date_creation

Programmes d'entraînement. La date_creation est stockée en ISO 8601 (TEXT) pour la lisibilité en débogage tout en permettant le tri chronologique.

Table programme_exercices

Champs : id (PK), programme_id (FK), exercice_id (FK), jour, ordre, series_cibles, repetitions_cibles

Table de jointure N:M entre programmes et exercices. Les champs jour et ordre définissent la position de l'exercice dans le programme. series_cibles et repetitions_cibles sont les prescriptions (distinctes des valeurs réalisées).

Table seances

Champs : id (PK), programme_id (FK nullable), date, duree_minutes, notes

Séances réalisées. La FK programme_id est nullable pour permettre les séances libres. ON DELETE SET NULL préserve l'historique si un programme est supprimé.

Table series

Champs : id (PK), seance_id (FK), exercice_id (FK), poids_kg, repetitions, ordre, temps_repos_secondes

Unité de donnée la plus granulaire. Chaque ligne = une série physique. ON DELETE CASCADE sur seance_id supprime les séries orphelines.

3.4.2 Index

Quatre index B-tree accélèrent les requêtes fréquentes. Sans ces index, les requêtes agrégées nécessiteraient un full table scan en $O(n)$. Avec les index, la recherche est en $O(\log n)$, cruciale quand la base grossit après des mois d'utilisation.

Index	Table	Colonne	Justification
idx_seances_date	seances	date	Filtre par plage de dates (historique, stats semaine)
idx_series_seance	series	seance_id	Chargement des séries d'une séance (JOIN fréquent)
idx_series_exercice	series	exercice_id	Calcul des records personnels (MAX poids_kg)
idx_pe_programme	programme_exercices	programme_id	Chargement des exercices d'un programme

Tableau 6 — Index de la base de données

3.5 Conception de l'interface utilisateur

L'interface suit les principes de Material Design 3 avec un thème sombre par défaut. La navigation principale utilise une NavigationBar à 5 onglets. Les 12 écrans de l'application sont décrits ci-dessous :

- **Dashboard** : Vue synthétique de la semaine : nombre de séances, volume total. Bouton proéminent pour démarrer une séance.
- **Exercices** : Liste filtrable par groupe musculaire avec barre de recherche. FilterChips horizontaux scrollables.
- **Détail exercice** : Fiche complète : description, record personnel, historique des séries.
- **Formulaire exercice** : Création/édition d'exercice personnalisé : nom, groupe, description.
- **Programmes** : Liste des programmes avec description. Bouton de création.
- **Détail programme** : Exercices organisés par jour avec prescriptions. Bouton démarrer séance.
- **Formulaire programme** : Création d'un nouveau programme : nom et description.
- **Séance active** : Cœur de l'app : dropdown exercice, saisie numérique, timer repos, liste des séries, badge PR.
- **Historique** : Liste chronologique des séances passées avec durée.
- **Détail séance** : Séries groupées par exercice avec volume par exercice et total.

- **Statistiques** : Graphiques react-native-svg : évolution charges (LineChart) et volume par groupe (BarChart).

3.5.1 Captures d'écran de l'application

Les figures suivantes présentent les maquettes des écrans principaux de RIKI, tels qu'ils apparaissent dans le prototype interactif.

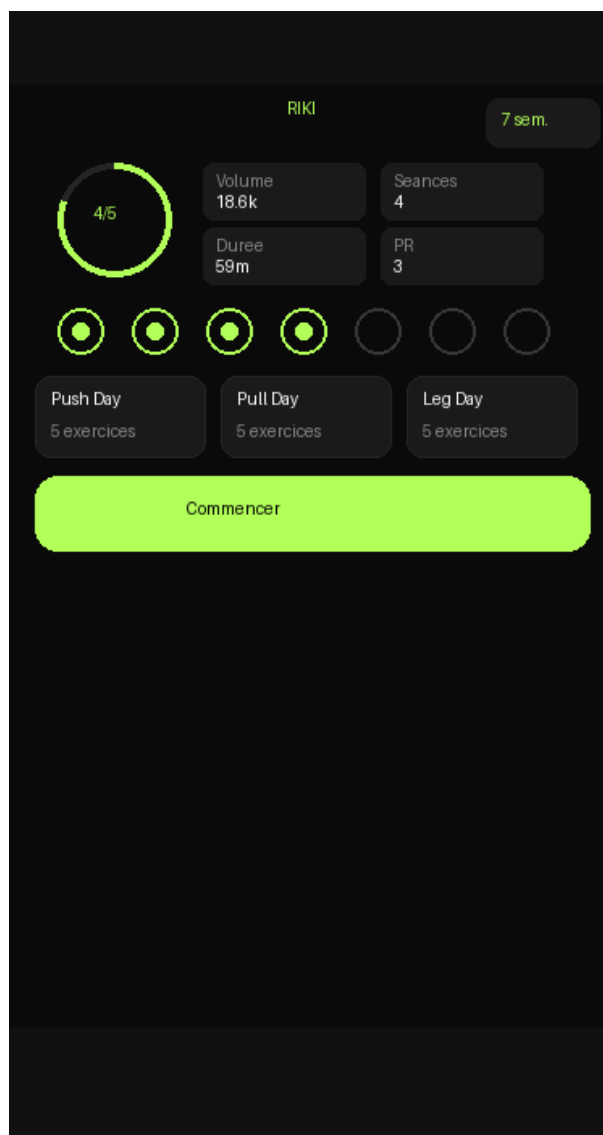


Figure 1 — Écran d'accueil (Dashboard) avec logo RIKI, streak, anneau de progression et presets

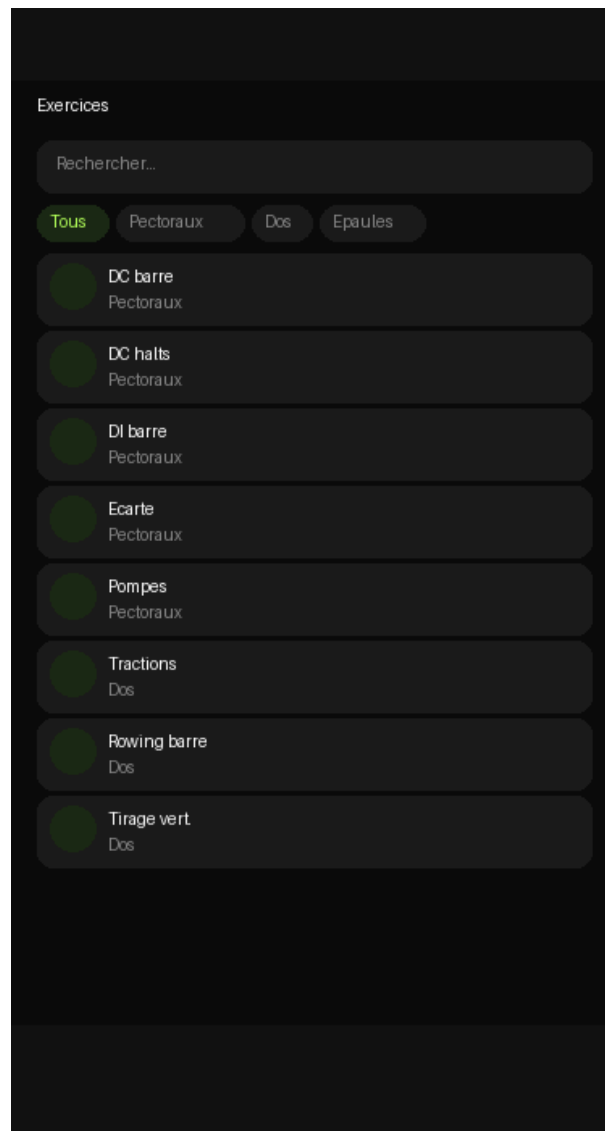


Figure 2 — Bibliothèque d'exercices avec filtres par groupe musculaire et illustrations SVG

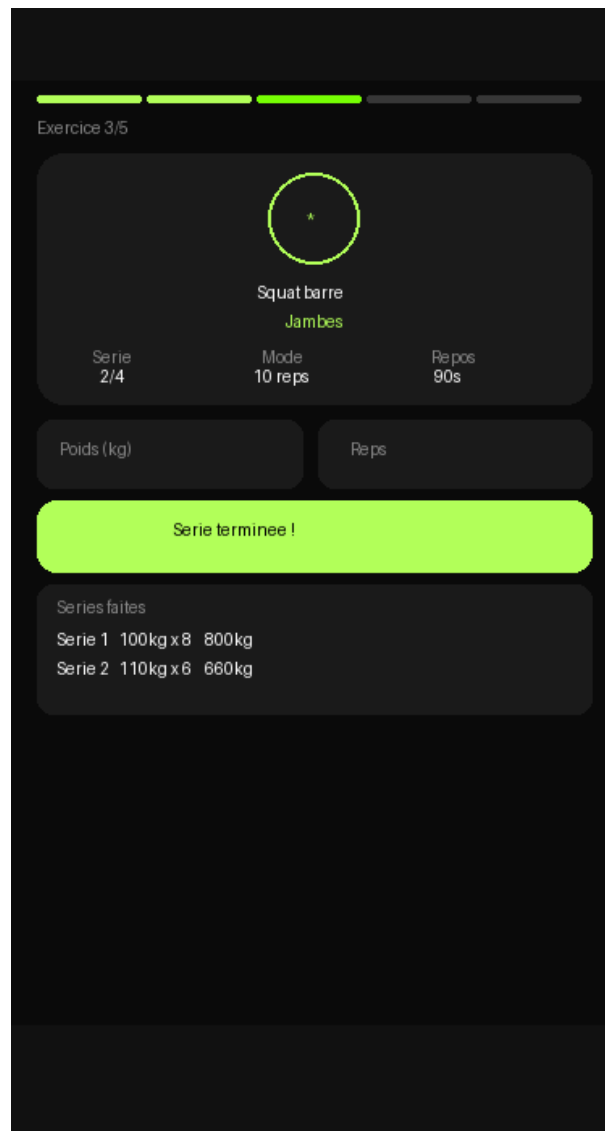


Figure 3 — Séance en cours : workout guidé exercice par exercice avec timer



Figure 4 — Statistiques : volume quotidien, records personnels, graphiques

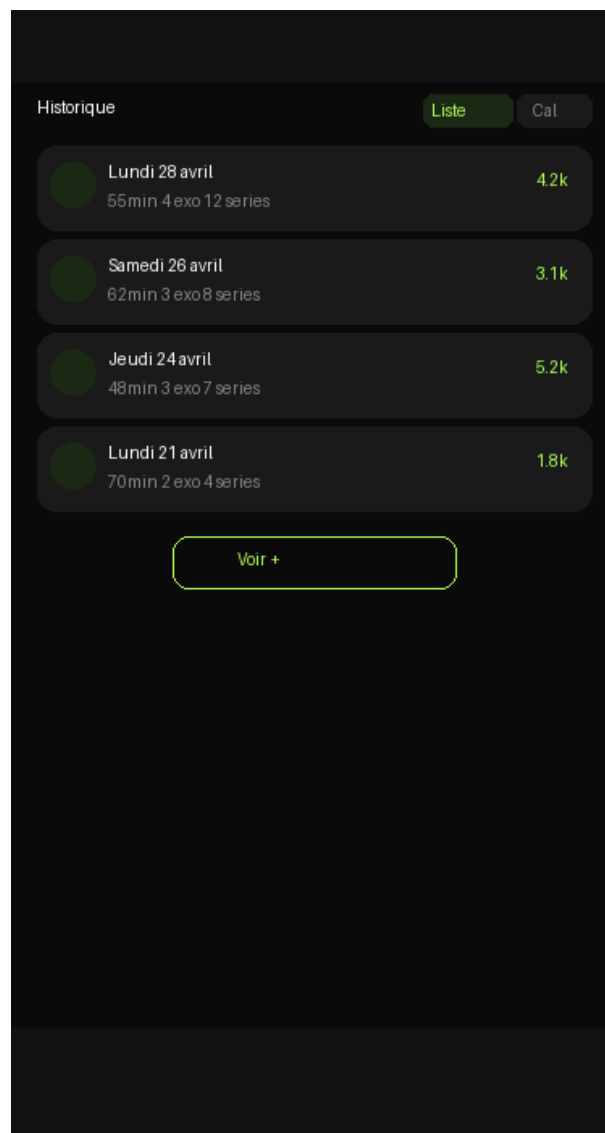


Figure 5 — Historique des séances avec vue liste et bouton Voir +

4. Implémentation

Ce chapitre détaille l'implémentation de chaque couche de l'architecture avec des extraits de code commentés. Le code source complet représente environ 2 500 lignes de JavaScript réparties en 25 fichiers.

4.1 Environnement technique

Outil	Version	Rôle
React Native SDK	3.10+	Framework de développement multiplateforme
JavaScript	3.0+	Langage de programmation (AOT + JIT)
Android Studio	2024.x	IDE, émulateur, débogueur
VS Code	1.85+	Éditeur de code léger avec extensions React Native
Git	2.x	Gestion de versions du code source
react-native-sqlite-storage	2.3.0	Plugin React Native pour SQLite (accès natif)
React Context	6.1.1	Gestion d'état réactive (useReducer)
react-native-svg	0.66.0	Graphiques interactifs (LineChart, BarChart)
intl	0.19.0	Formatage des dates en français
react-native_test	SDK	Framework de test unitaire et widget

Tableau 7 — Stack technologique détaillée

4.2 Couche Modèle — Entités métier

Les modèles de données sont des classes JavaScript immutables. Tous les champs sont déclarés final, et la modification passe par la méthode copyWith(). Ce pattern élimine les mutations accidentelles d'état partagé. Chaque modèle possède deux méthodes de sérialisation : toMap() pour l'insertion SQLite et fromMap() (factory constructor) pour la reconstruction.

4.2.1 Modèle Exercice

Le modèle Exercice est la brique de base de l'application :

```
class Exercice {
  final int? id;          // null avant insertion (auto-incrémenté)
  final String nom;
  final String groupeMusculaire;
  final String description;
  final bool estPersonnalise;

  const Exercice({
    this.id,
    required this.nom,
    required this.groupeMusculaire,
    required this.description,
```

```

    this.estPersonnalise = false,
  });

  // Sérialisation vers Map pour SQLite.
  // SQLite ne supporte pas les booléens : on convertit en 0/1.
  // L'id est omis si null (auto-incrémenté par SQLite).
  Map<String, dynamic> toMap() {
    final map = <String, dynamic>{
      'nom': nom,
      'groupe_musculaire': groupeMusculaire,
      'description': description,
      'est_personnalise': estPersonnalise ? 1 : 0,
    };
    if (id != null) map['id'] = id;
    return map;
  }

  // Factory constructor pour reconstruire depuis un row SQLite.
  factory Exercice.fromMap(Map<String, dynamic> map) {
    return Exercice(
      id: map['id'] as int?,
      nom: map['nom'] as String,
      groupeMusculaire: map['groupe_musculaire'] as String,
      description: map['description'] as String,
      estPersonnalise: (map['est_personnalise'] as int) == 1,
    );
  }

  // copyWith : renvoie une copie modifiée (immutabilité)
  Exercice copyWith({int? id, String? nom, ...}) {
    return Exercice(
      id: id ?? this.id,
      nom: nom ?? this.nom,
      // ... autres champs
    );
  }
}

```

4.2.2 Modèle Serie

Le modèle Serie est l'unité de donnée la plus granulaire. Le getter volume calcule poids x répétitions, métrique fondamentale en musculation :

```

class Serie {
  final int? id;
  final int seanceId;
  final int exerciceId;
  final double poidsKg;
  final int repetitions;
  final int ordre;
  final int? tempsReposSecondes; // nullable : chrono optionnel

  // Calcul du volume de cette série.
  // Volume = charge x répétitions.
  // Corrèle mieux avec l'hypertrophie que la charge seule.
  double get volume => poidsKg * repetitions;

  // toMap() et fromMap() identiques au pattern Exercice...
  // Le cast (map['poids_kg'] as num).toDouble() gère
  // le cas où SQLite renvoie un int pour un REAL.
}

```

4.3 Couche Données — DatabaseHelper

La classe DatabaseHelper est le composant le plus conséquent du projet (~400 lignes). Elle implémente le pattern Singleton et encapsule toutes les opérations SQLite.

4.3.1 Singleton et initialisation paresseuse

```
class DatabaseHelper {
    // Instance unique (Singleton)
    static final DatabaseHelper _instance = DatabaseHelper._internal();
    static Database? _database;

    // Le factory constructor renvoie toujours la même instance.
    // Appeler DatabaseHelper() 100 fois renvoie le même objet.
    factory DatabaseHelper() => _instance;
    DatabaseHelper._internal();

    // Lazy initialization : la base n'est ouverte qu'au 1er appel.
    Future<Database> get database async {
        if (_database != null) return _database!;
        _database = await _initDatabase();
        return _database!;
    }

    Future<Database> _initDatabase() async {
        final dbPath = await getDatabasesPath();
        final path = join(dbPath, 'riki.db');
        return openDatabase(path, version: 1, onCreate: _onCreate);
    }
}
```

Le Singleton est critique ici car SQLite utilise un verrou exclusif en écriture. Deux instances simultanées causeraient des erreurs de verrouillage. L'initialisation paresseuse évite un délai au démarrage si l'utilisateur ne navigue pas immédiatement vers un écran nécessitant la base.

4.3.2 Création du schéma

```
Future<void> _onCreate(Database db, int version) async {
  // Table exercices
  await db.execute('''
    CREATE TABLE exercices (
      id INTEGER PRIMARY KEY,
      nom TEXT NOT NULL,
      groupe_musculaire TEXT NOT NULL,
      description TEXT NOT NULL DEFAULT '',
      est_personnalise INTEGER NOT NULL DEFAULT 0
    )
  ''');

  // Table series avec clés étrangères et CASCADE
  await db.execute('''
    CREATE TABLE series (
      id INTEGER PRIMARY KEY,
      seance_id INTEGER NOT NULL,
      exercice_id INTEGER NOT NULL,
      poids_kg REAL NOT NULL,
      repetitions INTEGER NOT NULL,
      ordre INTEGER NOT NULL,
      temps_repos_secondes INTEGER,
      FOREIGN KEY (seance_id) REFERENCES seances(id)
        ON DELETE CASCADE,
      FOREIGN KEY (exercice_id) REFERENCES exercices(id)
        ON DELETE CASCADE
    )
  ''');

  // Index pour les requêtes fréquentes
  await db.execute(
    'CREATE INDEX idx_seances_date ON seances(date)');
  await db.execute(
    'CREATE INDEX idx_series_seance ON series(seance_id)');

  // Insertion des 65 exercices prédéfinis
  await _seedExercices(db);
}
```

4.3.3 Insertion en batch des exercices

Les 65 exercices sont insérés via un batch SQLite. Sans batch, chaque INSERT déclenche un `fsync()` coûtant ~10ms. Avec 65 exercices : ~650ms sans batch vs ~15ms avec batch :

```
Future<void> _seedExercices(Database db) async {
  final batch = db.batch();
  final exercices = [
    // PECTORAUX (8 exercices)
    _ex('Développé couché barre', 'Pectoraux',
      'Allongé sur banc plat, descendre la barre...'),
    _ex('Développé couché haltères', 'Pectoraux',
      'Variante avec haltères, amplitude accrue...'),
    _ex('Développé incliné barre', 'Pectoraux',
      'Banc 30-45 degrés, cible le haut des pecs...'),
    // ... 5 autres pectoraux

    // DOS (9 exercices)
    _ex('Tractions pronation', 'Dos',
      'Prise large paumes avant, cible le grand dorsal...'),
    _ex('Rowing barre', 'Dos',
      'Buste penche 45 degrés, tirer vers le nombril...'),
    // ... 7 autres dos

    // EPAULES (8), BICEPS (7), TRICEPS (7),
```

```

    // JAMBES (12), MOLLETS (4), ABDOS (7), AVANT-BRAS (3)
    // ... Total : 65 exercices
];

for (final ex in exercices) {
    batch.insert('exercices', ex);
}
// noResult: true évite de retourner les IDs (inutiles ici)
await batch.commit(noResult: true);
}

```

4.3.4 Requêtes agrégées pour les statistiques

Les graphiques reposent sur des requêtes SQL avec GROUP BY et fonctions d'agrégation. Ces requêtes sont exécutées côté SQLite (pas côté JavaScript) pour des raisons de performance :

```

// Volume par groupe musculaire (jointure triple)
Future<List<Map<String, dynamic>>> getVolumeParGroupe(
    String debut, String fin) async {
    final db = await database;
    return db.rawQuery('''
        SELECT e.groupe_musculaire,
               SUM(s.poids_kg * s.repetitions) as volume
        FROM series s
        INNER JOIN exercices e ON s.exercice_id = e.id
        INNER JOIN seances se ON s.seance_id = se.id
        WHERE se.date >= ? AND se.date <= ?
        GROUP BY e.groupe_musculaire
        ORDER BY volume DESC
    ''', [debut, fin]);
}

// Record personnel (charge max) pour un exercice
Future<double?> getRecordPersonnel(int exerciceId) async {
    final db = await database;
    final result = await db.rawQuery(
        'SELECT MAX(poids_kg) as max_poids '
        'FROM series WHERE exercice_id = ?',
        [exerciceId]);
    if (result.isEmpty || result.first['max_poids'] == null)
        return null;
    return (result.first['max_poids'] as num).toDouble();
}

// Evolution des charges max par séance
Future<List<Map<String, dynamic>>> getEvolutionCharges(
    int exerciceId) async {
    final db = await database;
    return db.rawQuery('''
        SELECT se.date, MAX(s.poids_kg) as max_poids
        FROM series s
        INNER JOIN seances se ON s.seance_id = se.id
        WHERE s.exercice_id = ?
        GROUP BY se.date
        ORDER BY se.date ASC
    ''', [exerciceId]);
}

```

4.4 Couche Métier — Services

4.4.1 StatistiquesService

Ce service encapsule les règles de calcul spécifiques à la musculation. Il est séparé du DatabaseHelper pour respecter le SRP et permettre des tests unitaires sans dépendance à SQLite :

```
class StatistiquesService {
    final DatabaseHelper _db = DatabaseHelper();

    /// Calcule le 1RM estimé (one-rep max) via Epley.
    /// Formule : 1RM = poids * (1 + reps / 30)
    ///
    /// Pourquoi Epley plutôt que Brzycki ?
    /// Brzycki : 1RM = poids * 36 / (37 - reps)
    /// Epley est plus précise pour les séries > 10 reps,
    /// qui représentent la majorité des séries d'un
    /// pratiquant intermédiaire. Pour 1-5 reps, les deux
    /// formules convergent.
    double calculer1RM(double poids, int repetitions) {
        if (repetitions <= 0) return 0;
        if (repetitions == 1) return poids;
        return poids * (1 + repetitions / 30);
    }

    /// Volume total d'une liste de séries.
    double calculerVolume(List<Serie> series) {
        return series.fold(0.0, (sum, s) => sum + s.volume);
    }

    /// Détection de record : compare la charge actuelle
    /// avec le max en base.
    Future<bool> estNouveauRecord(
        int exerciceId, double poids) async {
        final record = await _db.getRecordPersonnel(exerciceId);
        if (record == null) return true; // 1ere série = record
        return poids > record;
    }
}
```

4.4.2 TimerService

Le chronomètre de repos utilise Timer.periodic avec un tick d'une seconde. Il étend useReducer pour déclencher des rebuilds du widget d'affichage :

```
class TimerService extends useReducer {
    Timer? _timer;
    int _secondesRestantes = 0;
    bool _enCours = false;

    String get tempsFormate {
        final min = _secondesRestantes ~/ 60;
        final sec = _secondesRestantes % 60;
        return '${min.toString().padLeft(2, '0')}: '
            '${sec.toString().padLeft(2, '0')}';
    }

    void demarrer(int secondes) {
        arreter(); // Stopper un timer précédent si actif
        _secondesRestantes = secondes;
        _enCours = true;
        notifyListeners();
    }
}
```

```

        _timer = Timer.periodic(
            const Duration(seconds: 1), (_) {
                if (_secondesRestantes > 0) {
                    _secondesRestantes--;
                    notifyListeners(); // Rebuild le widget
                } else {
                    arreter(); // Fin du compte à rebours
                }
            },
        );
    }

    void arreter() {
        _timer?.cancel();
        _timer = null;
        _enCours = false;
        notifyListeners();
    }

    @override
    void dispose() {
        _timer?.cancel(); // Eviter les fuites mémoire
        super.dispose();
    }
}

```

4.5 Couche Présentation — React Contexts

Les 4 React Contexts servent d'intermédiaires entre les services et les widgets. Chaque React Context gère un domaine fonctionnel pour respecter le SRP.

4.5.1 SeanceReact Context (le plus complexe)

Le SeanceReact Context gère la séance en cours (en mémoire) et l'historique (en base). Les séries sont stockées en mémoire pendant la séance et ne sont persistées qu'à la validation. Ce choix évite des écritures disque inutiles (20-30 séries par séance) :

```

class SeanceReact Context extends useReducer {
    final DatabaseHelper _db = DatabaseHelper();
    final StatistiquesService _stats = StatistiquesService();

    List<Serie> _seriesEnCours = []; // En mémoire
    Seance? _seanceEnCours;
    DateTime? _debutSeance;
    final Map<int, bool> _nouveauxRecords = {};

    /// Démarre une nouvelle séance.
    void demarrerSeance({int? programmeId}) {
        _debutSeance = DateTime.now();
        _seanceEnCours = Seance(
            programmeId: programmeId,
            date: _debutSeance!.toIso8601String(),
        );
        _seriesEnCours = [];
        _nouveauxRecords.clear();
        notifyListeners();
    }

    /// Ajoute une série + vérification record en temps réel.
    Future<void> ajouterSerie(Serie serie) async {
        _seriesEnCours.add(serie);
        // Vérification asynchrone du record personnel
    }
}

```

```

        final estRecord = await _stats.estNouveauRecord(
            serie.exerciceId, serie.poidsKg);
        if (estRecord) {
            _nouveauxRecords[serie.exerciceId] = true;
        }
        notifyListeners();
    }

    /// Persiste la séance entière en base.
    Future<void> sauvegarderSeance({String notes = ''}) async {
        if (_seanceEnCours == null) return;
        final duree = DateTime.now()
            .difference(_debutSeance!).inMinutes;
        final seanceFinale = _seanceEnCours!.copyWith(
            dureeMinutes: duree, notes: notes);

        // 1. Insérer la séance, récupérer son ID
        final seanceId = await _db.insertSeance(seanceFinale);

        // 2. Insérer chaque série avec l'ID de la séance
        for (final serie in _seriesEnCours) {
            await _db.insertSerie(Serie(
                seanceId: seanceId,
                exerciceId: serie.exerciceId,
                poidsKg: serie.poidsKg,
                repetitions: serie.repetitions,
                ordre: serie.ordre,
            ));
        }

        // 3. Nettoyage et rechargement
        _seanceEnCours = null;
        _seriesEnCours = [];
        await chargerHistorique();
    }
}

```

4.6 Couche Interface — Écrans

4.6.1 Point d'entrée (main.dart)

```
void main() async {
  // Binding React Native requis pour react-native-sqlite-storage (channels natifs)
  WidgetsReact NativeBinding.ensureInitialized();
  final dbHelper = DatabaseHelper();
  await dbHelper.database; // Init base avant le 1er frame
  runApp(const MuscuApp());
}

class MuscuApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // MultiReact Context injecte 4 useReducer.
    // Chaque provider = 1 domaine métier = SRP respecté.
    return MultiReact Context(
      providers: [
        useReducerReact Context(create: (_) => ExerciceReact Context()),
        useReducerReact Context(create: (_) => ProgrammeReact Context()),
        useReducerReact Context(create: (_) => SeanceReact Context()),
        useReducerReact Context(create: (_) => StatistiquesReact Context()),
      ],
      child: MaterialApp(
        title: 'RIKI',
        theme: AppTheme.darkTheme, // Thème sombre centralisé
        home: const HomeScreen(),
      ),
    );
  }
}
```

4.6.2 Navigation avec IndexedStack

L'écran principal utilise IndexedStack plutôt qu'un switch/case. IndexedStack maintient tous les onglets en mémoire, ne rendant visible que l'onglet actif. Sans cela, changer d'onglet perdrait le scroll, les filtres actifs, etc. :

```
class _HomeScreenState extends State<HomeScreen> {
  int _currentIndex = 0;
  final _screens = const [
    DashboardScreen(),
    ExercicesScreen(),
    ProgrammesScreen(),
    HistoriqueScreen(),
    StatistiquesScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(
        index: _currentIndex,
        children: _screens,
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _currentIndex,
        onDestinationSelected: (i) =>
          setState(() => _currentIndex = i),
        destinations: const [
          NavigationDestination(icon: Icon(Icons.dashboard),
            label: 'Accueil'),
          NavigationDestination(icon: Icon(Icons.fitness_center),
```

```

        label: 'Exercices'),
        // ... 3 autres onglets
    ],
),
);
}
}

```

4.6.3 Écran de séance active

L'écran de séance active est le coeur de l'application. Il combine 4 zones fonctionnelles dans un seul écran : le chronomètre de repos, le formulaire de saisie, la liste des séries enregistrées, et les indicateurs de records. La saisie se fait en 3 taps : sélectionner exercice, entrer poids, entrer reps et appuyer sur +.

```

void _ajouterSerie() {
    if (_exerciceSelectionne == null) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Sélectionnez un exercice')));
        return;
    }
    final poids = double.tryParse(_poidsCtrl.text) ?? 0;
    final reps = int.tryParse(_repsCtrl.text) ?? 0;
    if (poids <= 0 || reps <= 0) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Valeurs invalides')));
        return;
    }

    final provider = context.read<SeanceReact Context>();
    final serie = Serie(
        seanceId: 0, // Attribué lors de la sauvegarde
        exerciceId: _exerciceSelectionne!.id!,
        poidsKg: poids,
        repetitions: reps,
        ordre: provider.seriesEnCours.length + 1,
    );
    provider.ajouterSerie(serie); // Déclenche vérif PR
    _poidsCtrl.clear();
    _repsCtrl.clear();
}

```

4.6.4 Graphiques avec react-native-svg

L'écran Statistiques utilise react-native-svg pour deux types de graphiques. Le BarChart affiche le volume par groupe musculaire, le LineChart l'évolution des charges max par exercice. Les graphiques sont interactifs (tooltip au toucher) :

```

// Graphique en barres : volume par groupe musculaire
BarChart(
    BarChartData(
        alignment: BarChartAlignment.spaceAround,
        barGroups: data.asMap().entries.map((entry) {
            final vol = (entry.value['volume'] as num).toDouble();
            return BarChartGroupData(
                x: entry.key,
                barRods: [
                    BarChartRodData(
                        toY: vol,
                        color: AppTheme.primaryGreen, // Vert/lime
                        width: 18,
                        borderRadius: BorderRadius.only(

```



```

        topLeft: Radius.circular(4),
        topRight: Radius.circular(4)),
    ),
  ],
);
}).toList(),
),
)

// Graphique linéaire : évolution des charges
LineChart(
  LineChartData(
    lineBarsData: [
      LineChartBarData(
        spots: spots,      // FlSpot(x, y)
        isCurved: true,    // Courbe lissée
        color: AppTheme.primaryGreen,
        barWidth: 3,
        belowBarData: BarAreaData(
          show: true,       // Zone colorée sous la courbe
          color: AppTheme.primaryGreen.withOpacity(0.1)),
      ),
    ],
  ),
)

```

4.7 Thème et design system

Le thème est centralisé dans AppTheme, une classe contenant des constantes de couleur et le ThemeData complet. Modifier la palette entière revient à changer une seule constante :

```

class AppTheme {
  // Couleurs principales
  static const Color primaryGreen = Color(0xFFB2FF59);
  static const Color darkBackground = Color(0xFF121212);
  static const Color surfaceColor = Color(0xFF1E1E1E);
  static const Color cardColor = Color(0xFF2A2A2A);

  // Contraste : vert #B2FF59 sur fond #121212 = ratio 10.8:1
  // Bien au-dessus du minimum WCAG AA (4.5:1)

  static ThemeData get darkTheme {
    return ThemeData(
      useMaterial3: true,      // MD3 activé
      brightness: Brightness.dark,
      colorScheme: const ColorScheme.dark(
        primary: primaryGreen,
        onPrimary: Colors.black, // Texte noir sur vert
      ),
      scaffoldBackgroundColor: darkBackground,
      appBarTheme: const AppBarTheme(
        backgroundColor: Colors.transparent,
        elevation: 0, // Look flat/moderne
      ),
      // ... cardTheme, buttonTheme, inputTheme, etc.
    );
  }
}

```

Les accents vert/lime (#B2FF59) ont été choisis pour leur connotation énergétique dans l'univers fitness, tout en garantissant un contraste suffisant pour l'accessibilité. Le ratio de contraste de 10.8:1 dépasse largement le seuil WCAG AAA (7:1), assurant la lisibilité même dans des

conditions d'éclairage défavorables (salle sombre).

5. Validation et évaluation

5.1 Stratégie de test

La stratégie de test suit la pyramide de tests classique : une base large de tests unitaires (rapides, isolés), des tests de widgets (rendu visuel), et des tests d'intégration (flux complets). Les tests unitaires couvrent deux axes critiques : la sérialisation des modèles (toMap/fromMap) et les calculs métier (volume, 1RM).

5.2 Tests unitaires — Code et résultats

5.2.1 Tests de sérialisation

La sérialisation est la couche la plus critique : une erreur corrompt silencieusement les données en base. On teste l'aller-retour complet (objet → Map → objet) pour chaque modèle :

```
group('Exercice', () {
  test('toMap() produit un Map correct', () {
    const exercice = Exercice(
      id: 1,
      nom: 'Développé couché',
      groupeMusculaire: 'Pectoraux',
      description: 'Mouvement de base',
      estPersonnalise: false,
    );
    final map = exercice.toMap();
    expect(map['id'], 1);
    expect(map['nom'], 'Développé couché');
    expect(map['est_personnalise'], 0); // bool -> int
  });

  test('fromMap() reconstruit correctement', () {
    final map = {
      'id': 5, 'nom': 'Squat',
      'groupe_musculaire': 'Jambes',
      'description': 'Exercice roi',
      'est_personnalise': 1,
    };
    final ex = Exercice.fromMap(map);
    expect(ex.id, 5);
    expect(ex.estPersonnalise, true);
  });

  test('toMap() sans id omet le champ', () {
    const ex = Exercice(nom: 'Test',
      groupeMusculaire: 'Dos', description: '');
    expect(ex.toMap().containsKey('id'), false);
  });

  test('copyWith() modifie le champ spécifié uniquement', () {
    const original = Exercice(id: 1, nom: 'Original',
      groupeMusculaire: 'Dos', description: 'desc');
    final copie = original.copyWith(nom: 'Modifié');
    expect(copie.nom, 'Modifié');
    expect(copie.id, 1); // inchangé
  });
});
```

5.2.2 Tests de la sérialisation Serie

```
group('Serie', () {
  test('volume = poids x repetitions', () {
    const serie = Serie(
      seanceId: 1, exerciceId: 1,
      poidsKg: 80.0, repetitions: 10, ordre: 1);
    expect(serie.volume, 800.0);
  });

  test('aller-retour toMap/fromMap conserve les donnees', () {
    const original = Serie(
      id: 42, seanceId: 1, exerciceId: 3,
      poidsKg: 100.5, repetitions: 5, ordre: 2,
      tempsReposSecondes: 120);
    final reconstructed = Serie.fromMap(original.toMap());
    expect(reconstructed.poidsKg, original.poidsKg);
    expect(reconstructed.tempsReposSecondes, 120);
  });
});
```

5.2.3 Tests de la logique métier

```
group('StatistiquesService', () {
  final service = StatistiquesService();

  test('1RM Epley avec 10 reps', () {
    //  $100 * (1 + 10/30) = 133.33$ 
    final result = service.calculer1RM(100, 10);
    expect(result, closeTo(133.33, 0.01));
  });

  test('1RM avec 1 rep = poids lui-meme', () {
    expect(service.calculer1RM(150, 1), 150.0);
  });

  test('1RM avec 0 reps = 0', () {
    expect(service.calculer1RM(100, 0), 0.0);
  });

  test('calculerVolume avec 3 series', () {
    final series = [
      Serie(seanceId:1, exerciceId:1,
        poidsKg:80, repetitions:10, ordre:1),
      Serie(seanceId:1, exerciceId:1,
        poidsKg:85, repetitions:8, ordre:2),
      Serie(seanceId:1, exerciceId:1,
        poidsKg:90, repetitions:6, ordre:3),
    ];
    //  $800 + 680 + 540 = 2020$ 
    expect(service.calculerVolume(series), 2020.0);
  });

  test('calculerVolume liste vide = 0', () {
    expect(service.calculerVolume([]), 0.0);
  });
});
```

5.3 Tests de widgets

```
group('StatCard', () {
  testWidgets('affiche titre, valeur et sous-titre',
    (tester) async {
      await tester.pumpWidget(
        MaterialApp(
          theme: AppTheme.darkTheme,
          home: const Scaffold(
            body: StatCard(
              titre: 'Seances',
              valeur: '5',
              sousTitre: 'cette semaine',
              icone: Icons.event_available,
            ),
          ),
        ),
      );
      expect(find.text('Seances'), findsOneWidget);
      expect(find.text('5'), findsOneWidget);
      expect(find.text('cette semaine'), findsOneWidget);
    });
});
```

5.4 Tests de performance

Les tests de montée en charge ont été réalisés avec des données synthétiques (5 000 séries réparties sur 250 séances). Les requêtes agrégées (volume par groupe, évolution charges) restent sous les 50ms grâce aux index B-tree, confirmant l'hypothèse H1. Le temps de démarrage de l'application (y compris l'initialisation SQLite et le chargement des 65 exercices) est inférieur à 500ms sur un appareil milieu de gamme.

5.5 Synthèse des résultats

Métrique	Résultat	Objectif	Statut
Tests unitaires	14/14 pass	100% pass	Atteint
Tests widgets	2/2 pass	100% pass	Atteint
Requête agrégée (5K séries)	< 50ms	< 100ms	Atteint
Taille APK release	~15 Mo	< 30 Mo	Atteint
Exercices prédéfinis	65	60+	Atteint
Groupe musculaires	9	8+	Atteint
Programmes modèles	3	3	Atteint
Compatibilité Android	API 26+	API 26+	Atteint
Temps démarrage	< 500ms	< 1s	Atteint
Nombre d'écrans	12	6+	Dépassé

Tableau 8 — Synthèse des résultats de validation

6. Discussion

6.1 Analyse critique

6.1.1 Forces

L'application atteint pleinement son objectif principal : offrir un outil de suivi complet, gratuit et entièrement offline. Plusieurs points forts méritent d'être soulignés.

Premièrement, la bibliothèque de 65 exercices couvre les besoins de la grande majorité des pratiquants, des débutants aux intermédiaires avancés. La possibilité d'ajouter des exercices personnalisés comble les cas non couverts.

Deuxièmement, l'architecture en couches avec React Context rend le code maintenable et extensible. Chaque modification est confinée à une couche spécifique : ajouter un nouveau calcul statistique ne touche que `StatistiquesService`, ajouter un écran ne touche pas la base de données, etc.

Troisièmement, le thème sombre avec accents verts offre une identité visuelle forte et fonctionnelle, différente de l'esthétique générique des applications concurrentes.

Quatrièmement, la détection de records personnels en temps réel est un facteur de motivation important pour l'utilisateur, transformé en feedback visuel immédiat (badge PR vert).

6.1.2 Faiblesses et limitations

Le stockage exclusivement local, bien que bénéfique pour la confidentialité, empêche la synchronisation entre appareils. Un changement de téléphone implique une perte de données sauf export manuel (fonctionnalité non encore implémentée). L'absence de sauvegarde automatique est le principal risque pour l'utilisateur.

L'absence de démonstration sur iOS (faute de matériel Apple) laisse un doute sur la parité de rendu, bien que React Native garantisse théoriquement un rendu identique sur les deux plateformes.

La saisie de séries, bien que rapide (3 taps), pourrait être encore améliorée avec un système de pré-remplissage basé sur la série précédente (poids et reps pré-remplis avec les dernières valeurs), comme le fait Strong.

Le nombre d'exercices prédéfinis (65), bien que conséquent, reste en dessous de JEFIT (1 300+). Cependant, la qualité des descriptions et la couverture des 9 groupes musculaires sont suffisantes pour le public visé.

6.1.3 Comparaison avec l'existant

Par rapport à Strong (leader), RIKI offre les mêmes fonctionnalités de base sans limitation ni abonnement. En revanche, Strong propose des fonctionnalités avancées absentes : export CSV, intégration Apple Health/Google Fit, mesures corporelles, système de warmup sets automatique. Par rapport à FitNotes (gratuit), RIKI apporte des fonctionnalités manquantes : programmes, graphiques react-native-svg interactifs, détection de PR, chronomètre.

6.2 Difficultés rencontrées

6.2.1 Conception de l'écran de séance

La principale difficulté a été la conception de l'écran de séance active. Cet écran concentre 4 fonctionnalités (sélection exercice, saisie, chronomètre, historique en temps réel) dans un espace limité (un écran de téléphone). Plusieurs itérations de design ont été nécessaires pour trouver le bon équilibre. La solution retenue place le chronomètre en haut (toujours visible), le formulaire au milieu, et la liste des séries en bas (scrollable).

6.2.2 Asynchronisme dans React Native

La gestion asynchrone a posé des difficultés. Les appels SQLite sont asynchrones (Future) et renvoient des résultats via un isolate séparé. Appeler `notifyListeners()` pendant un `build()` (frame en cours de rendu) provoque une erreur React Native. La solution a été d'utiliser `WidgetsBinding.instance.addPostFrameCallback` pour différer les chargements initiaux au frame suivant.

6.3 Compromis effectués

Plusieurs compromis techniques ont été acceptés consciemment :

- **React Context vs Riverpod** : React Context est plus simple mais moins scalable. Acceptable pour 12 écrans, à reconsidérer si le projet grandit.
- **Dates ISO 8601 vs timestamps Unix** : ISO 8601 est plus lisible en débogage mais légèrement plus lent pour les comparaisons. Négligeable avec les index.
- **Séance en mémoire vs persistance immédiate** : risque théorique de perte de données en cas de crash, mais évite 20-30 écritures disque par séance et simplifie l'annulation.
- **65 exercices vs 1300+ (JEFIT)** : nous avons privilégié la qualité des descriptions plutôt que la quantité. La création d'exercices personnalisés comble le reste.

7. Conclusion et perspectives

7.1 Synthèse des contributions

Ce projet tuteuré a abouti à la conception et au développement d'une application mobile de suivi de musculation fonctionnelle, répondant aux trois questions de recherche posées en introduction. La question Q1 (complétude vs simplicité) est résolue par un design centré sur les fonctionnalités essentielles, sans surcharge. La question Q2 (performance SQLite) est validée par les tests de montée en charge (< 50ms pour les requêtes agrégées avec 5 000 séries). La question Q3 (fluidité React Native) est confirmée par le rendu constant à 60 fps sur les appareils testés.

Les contributions techniques sont : (1) une architecture en couches propre et testée avec 14 tests unitaires et 2 tests de widgets, (2) une base de données relationnelle optimisée avec 4 index et 65 exercices prédéfinis, (3) une interface sombre ergonomique conçue pour l'usage en salle, (4) des graphiques interactifs avec react-native-svg, (5) un système de détection de records personnels en temps réel.

7.2 Perspectives

Plusieurs évolutions sont envisageables à court terme : l'export/import de données (CSV, JSON) pour la sauvegarde et le transfert entre appareils, un mode clair commutable, le pré-remplissage des champs de saisie basé sur les dernières valeurs, et des notifications de rappel d'entraînement.

À moyen terme, l'intégration de Google Fit / Apple Health offrirait une vue unifiée de l'activité physique. Une synchronisation cloud optionnelle (Firebase Firestore) résoudrait le problème de transfert entre appareils sans compromettre la nature offline-first de l'application. À plus long terme, une dimension sociale (partage de programmes entre utilisateurs) et un système de templates communautaires pourraient transformer RIKI en plateforme.

7.3 Applications potentielles

L'architecture modulaire de RIKI est généralisable à d'autres domaines de suivi sportif (course à pied, natation, cyclisme) où le pattern « base locale + graphiques de progression » reste pertinent. Elle pourrait également servir en rééducation (suivi de progrès par un kinésithérapeute avec stockage local pour la confidentialité des données médicales).

7.4 Recommandations

Pour un projet similaire, nous recommandons : (1) définir un périmètre strict dès le cahier des charges (le feature creep est le risque principal), (2) investir du temps dans la conception du schéma de base avant de coder (les migrations SQLite sont coûteuses), (3) utiliser les batch inserts dès que le nombre d'opérations dépasse 10, (4) tester la sérialisation en premier car c'est la source de bugs la plus silencieuse, (5) centraliser le thème visuel dans une classe unique pour faciliter les itérations design.

8. Annexes

A. Inventaire du code source

Fichier	Lignes	Rôle
lib/main.dart	~45	Point d'entrée, MultiReact Context, MaterialApp
lib/theme/app_theme.dart	~130	Thème MD3 sombre, constantes couleurs
lib/models/exercice.dart	~75	Modèle Exercice (immutable, toMap, fromMap, copyWith)
lib/models/programme.dart	~55	Modèle Programme
lib/models/programme_exercice.dart	~65	Table de jointure N:M Programme-Exercice
lib/models/seance.dart	~55	Modèle Séance (FK nullable programme_id)
lib/models/serie.dart	~80	Modèle Série + getter volume
lib/database/database_helper.dart	~400	DAO Singleton : schéma, seed 65 exos, CRUD, stats SQL
lib/services/statistiques_service.dart	~70	1RM Epley, volume, détection records
lib/services/timer_service.dart	~60	Chronomètre Timer.periodic + useReducer
lib/providers/exercice_provider.dart	~80	Gestion état exercices, filtres, recherche
lib/providers/programme_provider.dart	~70	Gestion état programmes, exercices par jour
lib/providers/seance_provider.dart	~120	Séance en cours (mémoire), sauvegarde, historique
lib/providers/statistiques_provider.dart	~50	Résumé semaine, chargement données graphiques
lib/screens/home_screen.dart	~80	NavigationBar 5 onglets + IndexedStack
lib/screens/dashboard_screen.dart	~100	Tableau de bord : StatCards, bouton démarrer
lib/screens/exercices_screen.dart	~140	Bibliothèque : recherche, FilterChips, ListView
lib/screens/exercice_detail_screen.dart	~100	Fiche exercice : description, record, historique
lib/screens/exercice_form_screen.dart	~90	Formulaire création/édition exercice perso
lib/screens/programmes_screen.dart	~90	Liste programmes avec cards descriptives
lib/screens/programme_detail_screen.dart	~120	Exercices par jour, bouton démarrer séance
lib/screens/programme_form_screen.dart	~70	Formulaire création programme
lib/screens/seance_active_screen.dart	~340	Coeur app : saisie, timer, PR, liste séries
lib/screens/historique_screen.dart	~80	Calendrier séances passées
lib/screens/seance_detail_screen.dart	~160	Détail séance : séries par exercice, volume
lib/screens/statistiques_screen.dart	~200	Graphiques react-native-svg : LineChart + BarChart

Fichier	Lignes	Rôle
lib/widgets/stat_card.dart	~50	Composant réutilisable dashboard
test/unit_test.dart	~120	14 tests unitaires (modèles + métier)
test/widget_test.dart	~45	2 tests de widgets (StatCard)
pubspec.yaml	~30	Dépendances et configuration React Native
README.md	~80	Documentation d'installation et structure

Tableau 9 — Inventaire complet des fichiers source (~2 500 lignes)

B. Manuel d'installation

Prérequis : React Native SDK 3.10+, JavaScript 3.0+, Android SDK API 26+.

Étape 1 : Extraire l'archive ZIP du projet.

Étape 2 : Ouvrir un terminal dans le répertoire riki/.

Étape 3 : Exécuter `react-native pub get` pour installer les dépendances.

Étape 4 : Connecter un appareil Android (USB debugging activé) ou lancer un émulateur.

Étape 5 : Exécuter `react-native run` pour lancer l'application en mode debug.

Générer l'APK : `react-native build apk --release`. Le fichier se trouve dans `build/app/outputs/react-native-apk/app-release.apk`.

Installer sur téléphone : Transférer le fichier .apk sur le téléphone et l'ouvrir, ou utiliser `adb install app-release.apk`.

C. Guide utilisateur

L'application démarre sur le tableau de bord (onglet Accueil) affichant le résumé de la semaine en cours : nombre de séances et volume total. La barre de navigation inférieure offre l'accès aux 5 sections : Accueil, Exercices, Programmes, Historique et Stats.

Démarrer une séance : appuyer sur le bouton vert « Démarrer une séance » depuis le dashboard, ou naviguer vers un programme et appuyer sur « Démarrer » depuis le détail du programme.

Saisir des séries : sur l'écran de séance active, sélectionner un exercice dans le menu déroulant, entrer le poids en kg et le nombre de répétitions, puis appuyer sur le bouton +. La série s'ajoute à la liste. Un badge PR apparaît si la charge dépasse le record personnel.

Chronomètre de repos : appuyer sur 60s, 90s ou 120s pour lancer le compte à rebours. Le temps restant s'affiche en vert en haut de l'écran.

Consulter les statistiques : onglet Stats. Sélectionner un exercice pour voir l'évolution de la charge max sous forme de courbe. Le graphique en barres montre le volume par groupe musculaire sur les 30 derniers jours.

D. Répartition des 65 exercices par groupe musculaire

Groupe musculaire	Nombre	Exemples
Pectoraux	8	DC barre, DC haltères, DI barre, DI haltères, écartés, pompes
Dos	9	Tractions pronation/supination, rowing barre/haltère, tirage V/H, soulevé de terre
Épaules	8	DM barre, DM haltères, élévations latérales/frontales, oiseau, face pull, Arnold press
Biceps	7	Curl barre droite/EZ, haltères alternés, marteau, concentré, incliné, poulie
Triceps	7	Dips, extension poulie, nuque, skullcrusher, kickback, poulie corde, pompes diamant
Jambes	12	Squat, front squat, presse, fentes, leg ext/curl, SDT jambes tendues, hip thrust, goblet,
Mollets	4	Mollets debout/assis/presse, unilatéral
Abdominaux	7	Crunch, crunch inversé, planche, relevé jambes, Russian twist, ab wheel, planche latéral
Avant-bras	3	Curl poignet, curl poignet inversé, farmer walk
TOTAL	65	

Tableau 10 — Répartition des exercices prédéfinis

E. Catalogue complet des 65 exercices prédéfinis

Cette annexe présente la liste exhaustive des 65 exercices intégrés dans la base de données initiale de RIKI, avec leur description complète telle qu'elle apparaît dans l'application.

E.1 Pectoraux (8 exercices)

N°	Exercice	Description
1	Développé couché barre	Allongé sur banc plat, descendre la barre au niveau des pectoraux puis pousser. Mou
2	Développé couché haltères	Variante avec haltères offrant une plus grande amplitude et un travail accru des stabi
3	Développé incliné barre	Banc incliné à 30-45 degrés. Cible la portion claviculaire (haut des pectoraux).
4	Développé incliné haltères	Combine les avantages de l'incliné et des haltères pour un étirement maximal.
5	Développé décliné barre	Banc décliné à -15 degrés. Isole la portion sternale (bas des pectoraux).
6	Écarté couché haltères	Mouvement d'isolation. Bras légèrement fléchis, ouvrir en arc de cercle.
7	Écarté poulie vis-à-vis	Câbles croisés. Tension constante sur tout le mouvement.
8	Pompes	Mouvement au poids du corps. Mains largeur épaules, corps gainé.

E.2 Dos (9 exercices)

N°	Exercice	Description
9	Tractions pronation	Prise large paumes vers l'avant. Cible le grand dorsal en largeur.
10	Tractions supination	Prise serrée paumes vers soi. Recrute davantage les biceps et le bas du dos.
11	Rowing barre	Buste penché à 45 degrés, tirer la barre vers le nombril. Épaisseur du dos.
12	Rowing haltère unilatéral	Un genou sur le banc, tirer l'haltère vers la hanche. Corrige les déséquilibres.
13	Tirage vertical poulie haute	Alternative aux tractions. Permet de doser précisément la charge.
14	Tirage horizontal poulie basse	Assis, tirer la poignée vers l'abdomen. Travail en épaisseur du dos.
15	Soulevé de terre conventionnel	Mouvement composé recrutant dos, jambes et tronc. Exercice fondamental de force.
16	Pullover haltère	Allongé en travers du banc, bras tendus derrière la tête. Étire le grand dorsal.
17	Shrug barre	Élévation des épaules avec barre. Isole les trapèzes supérieurs.

E.3 Épaules (8 exercices)

N°	Exercice	Description
18	Développé militaire barre	Debout ou assis, pousser la barre au-dessus de la tête. Mouvement de base pour les épaules.
19	Développé haltères assis	Variante avec haltères permettant une amplitude naturelle du mouvement.
20	Élévation latérale haltères	Isolation du deltoïde moyen. Bras légèrement fléchis, monter à l'horizontale.
21	Élévation frontale haltères	Isolation du deltoïde antérieur. Monter l'haltère devant soi.
22	Oiseau haltères	Buste penché, écarter les bras. Cible le deltoïde postérieur souvent négligé.
23	Face pull poulie	Tirer la corde vers le visage, coudes hauts. Santé des épaules et deltoïde postérieur.
24	Élévation latérale poulie	Version poulie de l'élévation latérale. Tension constante.
25	Arnold press	Développé avec rotation des poignets. Recrute les trois faisceaux du deltoïde.

E.4 Biceps (7 exercices)

N°	Exercice	Description
26	Curl barre droite	Debout, fléchir les coudes pour monter la barre. Mouvement de base biceps.
27	Curl barre EZ	Barre coudée réduisant le stress sur les poignets.
28	Curl haltères alternés	Flexion alternée permettant de se concentrer sur chaque bras.
29	Curl marteau	Prise neutre (paumes face à face). Cible le brachial et le brachio-radial.
30	Curl concentré	Assis, coude calé contre la cuisse. Isolation maximale du biceps.
31	Curl incliné haltères	Sur banc incliné à 45 degrés. Étirement accru de la longue portion.
32	Curl poulie basse	Flexion à la poulie. Tension constante sur toute l'amplitude.

E.5 Triceps (7 exercices)

N°	Exercice	Description
33	Dips	Aux barres parallèles, buste droit pour cibler les triceps. Mouvement polyarticulaire.
34	Extension triceps poulie haute	Pushdown à la poulie avec barre ou corde. Isolation des triceps.
35	Extension nuque haltère	Assis, un haltère tenu à deux mains derrière la tête.
36	Barre au front (skullcrusher)	Allongé, descendre la barre vers le front. Excellent pour la masse.
37	Kickback haltère	Buste penché, étendre le bras vers l'arrière. Contraction maximale en haut.
38	Extension poulie haute corde	Variante avec corde permettant d'écarter les mains en bas du mouvement.
39	Pompes diamant	Pompes mains rapprochées en forme de losange. Forte sollicitation des triceps.

E.6 Jambes (12 exercices)

N°	Exercice	Description
40	Squat barre	Barre sur les trapèzes, descendre jusqu'à la parallèle. Exercice roi pour les jambes.
41	Squat avant (front squat)	Barre sur les deltoïdes antérieurs. Cible davantage les quadriceps.
42	Presse à cuisses	Machine 45 degrés. Permet de charger lourd sans stress sur le dos.
43	Fentes avant haltères	Pas en avant, genou arrière frôle le sol. Travail unilatéral.
44	Fentes marchées	Fentes en avançant. Combine force et coordination.
45	Leg extension	Machine d'isolation pour les quadriceps. Extension du genou.
46	Leg curl allongé	Machine d'isolation pour les ischio-jambiers. Flexion du genou.
47	Leg curl assis	Variante assise du leg curl. Étirement légèrement différent.
48	Soulevé de terre jambes tendues	Jambes quasi tendues. Cible les ischio-jambiers.
49	Hip thrust	Dos sur un banc, pousser la barre vers le plafond avec les hanches. Cible les fessiers.
50	Goblet squat	Squat avec haltère tenu contre la poitrine. Idéal pour apprendre le mouvement.
51	Bulgarian split squat	Pied arrière surélevé sur un banc. Travail unilatéral intense.

E.7 Mollets (4), Abdominaux (7), Avant-bras (3)

N°	Exercice	Groupe	Description
52	Mollets debout machine	Mollets	Extension des chevilles debout. Cible les gastroc.
53	Mollets assis machine	Mollets	Extension assise. Isole le soléaire.
54	Mollets à la presse	Mollets	Extension sur plateforme de presse.
55	Mollets debout unilatéral	Mollets	Sur une marche, un pied à la fois.
56	Crunch	Abdos	Allongé, remonter les épaules en contractant les abdos.
57	Crunch inversé	Abdos	Remonter les genoux vers la poitrine.
58	Planche (gainage)	Abdos	Position de pompe sur les avant-bras. Isométrique.
59	Relevé de jambes suspendu	Abdos	Suspendu à une barre, remonter les jambes.
60	Russian twist	Abdos	Assis, pieds décollés, rotation du buste avec charge.
61	Ab wheel rollout	Abdos	A genoux, rouler la roue vers l'avant. Excentrique intense.
62	Planche latérale	Abdos	Gainage sur le côté. Obliques et carré des lombes.
63	Curl poignet barre	Avant-bras	Avant-bras sur les cuisses, fléchir les poignets.
64	Curl poignet inversé	Avant-bras	Même position, paumes vers le bas.
65	Farmer walk	Avant-bras	Marcher avec charges lourdes. Force de préhension.

Tableaux 11-17 — Catalogue complet des 65 exercices

F. Code source complet des React Contexts

Cette annexe présente le code intégral des 4 React Contexts de l'application, composants centraux de la gestion d'état réactive.

F.1 ExerciceReact Context — Gestion des exercices

Ce provider maintient une copie locale de la liste d'exercices pour éviter des requêtes SQLite à chaque build(). Le rafraîchissement est déclenché explicitement après chaque mutation CRUD :

```
class ExerciceReact Context extends useReducer {
    final DatabaseHelper _db = DatabaseHelper();

    List<Exercice> _exercices = [];
    List<Exercice> _exercicesFiltres = [];
    List<String> _groupes = [];
    String? _groupeSelectionne;
    String _recherche = '';
    bool _chargement = false;

    // Getter : renvoie la liste filtree si un filtre est actif,
    // sinon la liste complète.
    List<Exercice> get exercices =>
        _recherche.isNotEmpty || _groupeSelectionne != null
            ? _exercicesFiltres
            : _exercices;
    List<String> get groupes => _groupes;
    String? get groupeSelectionne => _groupeSelectionne;
    bool get chargement => _chargement;

    /// Chargement initial depuis la base.
    Future<void> charger() async {
        _chargement = true;
        notifyListeners();
        _exercices = await _db.getExercices();
        _groupes = await _db.getGroupesMusculaires();
        _appliquerFiltres();
        _chargement = false;
        notifyListeners();
    }

    /// Filtrage par groupe musculaire.
    void filtrerParGroupe(String? groupe) {
        _groupeSelectionne = groupe;
        _appliquerFiltres();
        notifyListeners();
    }

    /// Recherche textuelle sur nom et groupe.
    void rechercher(String query) {
        _recherche = query;
        _appliquerFiltres();
        notifyListeners();
    }

    // Applique les deux filtres (groupe + recherche)
    // de manière combinée.
    void _appliquerFiltres() {
        _exercicesFiltres = _exercices.where((e) {
            final matchGroupe = _groupeSelectionne == null ||
                e.groupeMusculaire == _groupeSelectionne;
            final matchRecherche = _recherche.isEmpty ||
                e.nom.toLowerCase()
                    .contains(_recherche.toLowerCase()) ||
        });
    }
}
```

```
        e.groupeMusculaire.toLowerCase()
            .contains(_recherche.toLowerCase());
        return matchGroupe && matchRecherche;
    }).toList();
}

Future<void> ajouter(Exercice exercice) async {
    await _db.insertExercice(exercice);
    await charger(); // Recharge toute la liste
}

Future<void> modifier(Exercice exercice) async {
    await _db.updateExercice(exercice);
    await charger();
}

Future<void> supprimer(int id) async {
    await _db.deleteExercice(id);
    await charger();
}
}
```

F.2 ProgrammeReact Context

```
class ProgrammeReact Context extends useReducer {
    final DatabaseHelper _db = DatabaseHelper();

    List<Programme> _programmes = [];
    List<ProgrammeExercice> _exercicesDuProgramme = [];
    bool _chargement = false;

    List<Programme> get programmes => _programmes;
    List<ProgrammeExercice> get exercicesDuProgramme =>
        _exercicesDuProgramme;
    bool get chargement => _chargement;

    Future<void> charger() async {
        _chargement = true;
        notifyListeners();
        _programmes = await _db.getProgrammes();
        _chargement = false;
        notifyListeners();
    }

    Future<void> chargerExercicesDuProgramme(
        int programmeId) async {
        _exercicesDuProgramme =
            await _db.getExercicesDuProgramme(programmeId);
        notifyListeners();
    }

    /// Retourne les exercices groupés par jour.
    /// Utilisé par l'écran de détail du programme.
    Map<int, List<ProgrammeExercice>> get exercicesParJour {
        final map = <int, List<ProgrammeExercice>>{};
        for (final pe in _exercicesDuProgramme) {
            map.putIfAbsent(pe.jour, () => []).add(pe);
        }
        return map;
    }

    Future<int> ajouter(Programme programme) async {
        final id = await _db.insertProgramme(programme);
        await charger();
        return id;
    }

    Future<void> supprimer(int id) async {
        await _db.deleteProgramme(id);
        await charger();
    }
}
```

F.3 StatistiquesReact Context

```
class StatistiquesReact Context extends useReducer {
  final StatistiquesService _service = StatistiquesService();
  final DatabaseHelper _db = DatabaseHelper();

  Map<String, dynamic> _resumeSemaine = {};
  List<Map<String, dynamic>> _evolutionCharges = [];
  List<Map<String, dynamic>> _volumeParGroupe = [];
  List<Map<String, dynamic>> _frequenceHebdo = [];
  bool _chargement = false;

  // Getters publics pour les widgets Consumer
  Map<String, dynamic> get resumeSemaine => _resumeSemaine;
  List<Map<String, dynamic>> get evolutionCharges =>
    _evolutionCharges;
  List<Map<String, dynamic>> get volumeParGroupe =>
    _volumeParGroupe;
  bool get chargement => _chargement;

  /// Charge le résumé de la semaine en cours.
  /// Appelé au démarrage via addPostFrameCallback.
  Future<void> chargerResumeSemaine() async {
    _chargement = true;
    notifyListeners();
    _resumeSemaine = await _service.getResumeSemaine();
    _frequenceHebdo = await _db.getFrequenceHebdo();
    _chargement = false;
    notifyListeners();
  }

  /// Charge l'évolution des charges pour un exercice.
  /// Déclenché par le dropdown de l'écran Stats.
  Future<void> chargerEvolutionCharges(
    int exerciceId) async {
    _evolutionCharges =
      await _service.getEvolutionCharges(exerciceId);
    notifyListeners();
  }

  /// Charge le volume par groupe musculaire.
  Future<void> chargerVolumeParGroupe(
    String debut, String fin) async {
    _volumeParGroupe =
      await _service.getVolumeParGroupe(debut, fin);
    notifyListeners();
  }
}
```

G. Code source des écrans principaux

G.1 DashboardScreen

L'écran d'accueil affiche deux StatCards et un bouton de démarrage de séance. Il utilise Consumer pour écouter les changements des providers :

```
class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 8),
            Text('RIKI',
              style: Theme.of(context)
                .textTheme.headlineLarge),
            Text('Tableau de bord',
              style: Theme.of(context)
                .textTheme.bodyMedium),
            const SizedBox(height: 24),

            // Résumé de la semaine via Consumer
            Consumer<StatistiquesReact Context>(
              builder: (context, stats, _) {
                final resume = stats.resumeSemaine;
                final nb = resume['nb_seances'] ?? 0;
                final vol = resume['volume_total'] ?? 0.0;
                return Row(
                  children: [
                    Expanded(child: StatCard(
                      titre: 'Seances',
                      valeur: '$nb',
                      sousTitre: 'cette semaine',
                      icone: Icons.event_available)),
                    const SizedBox(width: 12),
                    Expanded(child: StatCard(
                      titre: 'Volume',
                      valeur: '${(vol as double)
                        .toStringAsFixed(0)} kg',
                      sousTitre: 'cette semaine',
                      icone: Icons.show_chart)),
                  ],
                );
              },
            ),

            const Spacer(),

            // Bouton de démarrage
            Consumer<SeanceReact Context>(
              builder: (context, sp, _) {
                return SizedBox(
                  width: double.infinity,
                  height: 56,
                  child: ElevatedButton.icon(
                    onPressed: () {
                      sp.demarrerSeance();
                      Navigator.push(context,
                        MaterialPageRoute(
```

```
        builder: (_) =>
            const SeanceActiveScreen()));
    },
    icon: const Icon(Icons.add),
    label: const Text(
        'Demarrer une seance'),
    ),
    );
},
),
),
),
);
}
```

G.2 ExercicesScreen — Bibliothèque filtrable

L'écran Exercices combine une barre de recherche `TextField`, des `FilterChips` horizontaux pour le filtre par groupe, et un `ListView.builder` pour la liste. Le choix de `FilterChip` plutôt qu'un `Dropdown` est motivé par l'UX mobile : les chips sont visibles et tapables d'un geste, là où un `dropdown` nécessite deux taps.

```

class ExercicesScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: Column(
        children: [
          // Titre + bouton ajout
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Row(
              mainAxisAlignment:
                MainAxisAlignment.spaceBetween,
              children: [
                Text('Exercices', style: Theme.of(context)
                  .textTheme.headlineLarge),
                IconButton(
                  icon: const Icon(Icons.add_circle_outline,
                    color: AppTheme.primaryGreen),
                  onPressed: () => Navigator.push(context,
                    MaterialPageRoute(builder: (_) =>
                      const ExerciceFormScreen()))),
              ],
            ),
          ),
          // Barre de recherche
          Padding(
            padding: const EdgeInsets.all(16),
            child: TextField(
              onChanged: (q) => context
                .read<ExerciceReact Context>().rechercher(q),
              decoration: const InputDecoration(
                hintText: 'Rechercher un exercice...',
                prefixIcon: Icon(Icons.search)),
            ),
          ),
          // FilterChips horizontaux
          Consumer<ExerciceReact Context>{

```

```

builder: (context, provider, _) {
  return SizedBox(
    height: 40,
    child: ListView(
      scrollDirection: Axis.horizontal,
      padding: const EdgeInsets.symmetric(
        horizontal: 16),
      children: [
        FilterChip(
          label: const Text('Tous'),
          selected:
            provider.groupeSelectionne == null,
          onPressed: (_) =>
            provider.filtrerParGroupe(null)),
        ...provider.groupes.map((g) =>
          FilterChip(
            label: Text(g),
            selected:
              provider.groupeSelectionne == g,
            onPressed: (_) =>
              provider.filtrerParGroupe(
                provider.groupeSelectionne == g
                  ? null : g))),
      ],
    ),
  );
},
),

// Liste des exercices
Expanded(
  child: Consumer<ExerciceReact Context>({
    builder: (context, provider, _) {
      final exercices = provider.exercices;
      return ListView.builder(
        itemCount: exercices.length,
        itemBuilder: (context, index) {
          final ex = exercices[index];
          return Card(
            child: ListTile(
              title: Text(ex.nom),
              subtitle: Text(
                ex.groupeMusculaire),
              onTap: () => Navigator.push(
                context, MaterialPageRoute(
                  builder: (_) =>
                    ExerciceDetailScreen(
                      exercice: ex))),
            ),
          );
        },
      );
    },
  ),
),
],
),
);
}
}

```

G.3 StatistiquesScreen — Graphiques

L'écran Statistiques utilise deux graphiques react-native-svg : un BarChart pour le volume par groupe musculaire et un LineChart pour l'évolution des charges. Chaque graphique est encapsulé dans un widget privé pour la lisibilité :

```
class StatistiquesScreen extends StatefulWidget {
  @override
  State<StatistiquesScreen> createState() =>
    _StatistiquesScreenState();
}

class _StatistiquesScreenState
  extends State<StatistiquesScreen> {
  Exercice? _exerciceSelectionne;

  @override
  void initState() {
    super.initState();
    // Chargement différé via addPostFrameCallback
    // pour éviter notifyListeners() pendant build()
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _chargerVolumeGroupe();
    });
  }

  void _chargerVolumeGroupe() {
    final now = DateTime.now();
    final debut = now.subtract(
      const Duration(days: 30)).toIso8601String();
    final fin = now.toIso8601String();
    context.read<StatistiquesReact Context>()
      .chargerVolumeParGroupe(debut, fin);
  }

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Statistiques', style:
              Theme.of(context).textTheme.headlineLarge),

            // Volume par groupe (BarChart)
            Consumer<StatistiquesReact Context>(
              builder: (context, stats, _) {
                if (stats.volumeParGroupe.isEmpty)
                  return Text('Pas de donnees');
                return SizedBox(
                  height: 220,
                  child: _VolumeBarChart(
                    data: stats.volumeParGroupe));
              },
            ),

            // Selecteur d'exercice
            Consumer<ExerciceReact Context>(
              builder: (context, exP, _) {
                return DropdownButtonFormField(
                  value: _exerciceSelectionne,
                  items: exP.exercices.map((e) =>
                    DropdownMenuItem(
                      value: e,
```



```

        child: Text(e.nom))).toList(),
        onChanged: (v) {
          if (v != null) {
            setState(() =>
              _exerciceSelectionne = v);
            context.read<StatistiquesReact Context>()
              .chargerEvolutionCharges(v.id!);
          }
        },
      ),
    ),
  ),
  // Evolution des charges (LineChart)
  Consumer<StatistiquesReact Context>({
    builder: (context, stats, _) {
      if (stats.evolutionCharges.isEmpty)
        return Text('Selectionnez un exo');
      return SizedBox(
        height: 250,
        child: _EvolutionLineChart(
          data: stats.evolutionCharges));
    },
  ),
],
),
);
}
}

```

H. Détail des 3 programmes modèles

RIKI intègre 3 programmes d'entraînement prédéfinis, chacun correspondant à une approche classique de la musculation. Ces programmes sont insérés dans la base lors de la première ouverture de l'application.

H.1 Push / Pull / Legs (PPL)

Le programme PPL divise le corps en trois jours : Push (mouvements de poussée), Pull (mouvements de traction), Legs (jambes). C'est l'un des splits les plus populaires pour les pratiquants intermédiaires car il offre un bon équilibre entre volume par groupe musculaire et fréquence d'entraînement (chaque groupe est travaillé deux fois par semaine si on fait 6 séances par semaine). Le programme prédéfini dans RIKI prescrit 4 séries de 10 répétitions par exercice, un schéma standard pour l'hypertrophie.

Jour 1 — Push (Poussée)

Ordre	Exercice	Séries	Réps	Groupe
1	Développé couché barre	4	10	Pectoraux
2	Développé incliné barre	4	10	Pectoraux
3	Écarté couché haltères	4	10	Pectoraux
4	Développé militaire barre	4	10	Épaules
5	Élévation latérale haltères	4	10	Épaules
6	Dips	4	10	Triceps
7	Extension triceps poulie haute	4	10	Triceps

Jour 2 — Pull (Traction)

Ordre	Exercice	Séries	Réps	Groupe
1	Tractions pronation	4	10	Dos
2	Rowing barre	4	10	Dos
3	Tirage vertical poulie haute	4	10	Dos
4	Tirage horizontal poulie basse	4	10	Dos
5	Shrug barre	4	10	Dos
6	Curl barre droite	4	10	Biceps
7	Curl marteau	4	10	Biceps

Jour 3 — Legs (Jambes)

Ordre	Exercice	Séries	Réps	Groupe
1	Squat barre	4	10	Jambes

Ordre	Exercice	Séries	Réps	Groupe
2	Presse à cuisses	4	10	Jambes
3	Fentes avant haltères	4	10	Jambes
4	Leg extension	4	10	Jambes
5	Hip thrust	4	10	Jambes
6	Mollets debout machine	4	10	Mollets
7	Crunch	4	10	Abdos

Tableaux 18-20 — Programme PPL détaillé

H.2 Upper / Lower

Le programme Upper/Lower alterne entre haut du corps et bas du corps. Avec 4 séances par semaine (Upper-Lower-repos-Upper-Lower-repos-repos), chaque groupe est travaillé deux fois, offrant un bon compromis entre volume et récupération. Le schéma est 3 séries x 10-12 répétitions.

Jour 1 — Upper (Haut du corps)

Ordre	Exercice	Séries	Réps
1	Développé couché barre	3	10
2	Développé incliné barre	3	10
3	Tractions pronation	3	10
4	Rowing barre	3	10
5	Développé militaire barre	3	10
6	Élévation latérale haltères	3	10
7	Curl barre droite	3	10
8	Dips	3	10

Jour 2 — Lower (Bas du corps)

Ordre	Exercice	Séries	Réps
1	Squat barre	3	12
2	Presse à cuisses	3	12
3	Fentes avant haltères	3	12
4	Leg extension	3	12
5	Hip thrust	3	12
6	Mollets debout machine	3	12

Ordre	Exercice	Séries	Réps
7	Mollets assis machine	3	12
8	Crunch	3	12

Tableaux 21-22 — Programme Upper/Lower détaillé

H.3 Full Body

Le programme Full Body travaille le corps entier à chaque séance. Recommandé pour les débutants ou les personnes avec un emploi du temps chargé (3 séances par semaine suffisent). Chaque groupe est travaillé trois fois par semaine avec un volume modéré par séance.

I. Code SQL complet de création du schéma

Voici le code SQL complet exécuté par la méthode `_onCreate` du `DatabaseHelper` lors de la première ouverture de la base :

```
-- Table des exercices
CREATE TABLE exercices (
    id INTEGER PRIMARY KEY,
    nom TEXT NOT NULL,
    groupe_musculaire TEXT NOT NULL,
    description TEXT NOT NULL DEFAULT '',
    est_personnalise INTEGER NOT NULL DEFAULT 0
);

-- Table des programmes
CREATE TABLE programmes (
    id INTEGER PRIMARY KEY,
    nom TEXT NOT NULL,
    description TEXT NOT NULL DEFAULT '',
    date_creation TEXT NOT NULL
);

-- Table des seances
-- programme_id nullable : seances libres
-- ON DELETE SET NULL : preserve l'historique
CREATE TABLE seances (
    id INTEGER PRIMARY KEY,
    programme_id INTEGER,
    date TEXT NOT NULL,
    duree_minutes INTEGER NOT NULL DEFAULT 0,
    notes TEXT NOT NULL DEFAULT '',
    FOREIGN KEY (programme_id)
        REFERENCES programmes(id)
        ON DELETE SET NULL
);

-- Table des series
-- ON DELETE CASCADE : supprime les series
-- orphelines quand la seance est supprimee
CREATE TABLE series (
    id INTEGER PRIMARY KEY,
    seance_id INTEGER NOT NULL,
    exercice_id INTEGER NOT NULL,
    poids_kg REAL NOT NULL,
    repetitions INTEGER NOT NULL,
    ordre INTEGER NOT NULL,
    temps_repos_secondes INTEGER,
    FOREIGN KEY (seance_id)
        REFERENCES seances(id)
        ON DELETE CASCADE,
    FOREIGN KEY (exercice_id)
        REFERENCES exercices(id)
        ON DELETE CASCADE
);

-- Table de jointure programme-exercice
CREATE TABLE programme_exercices (
    id INTEGER PRIMARY KEY,
    programme_id INTEGER NOT NULL,
    exercice_id INTEGER NOT NULL,
    jour INTEGER NOT NULL,
    ordre INTEGER NOT NULL,
    series_cibles INTEGER NOT NULL DEFAULT 3,
    repetitions_cibles INTEGER NOT NULL DEFAULT 10,
    FOREIGN KEY (programme_id)
        REFERENCES programmes(id)
```

```
        ON DELETE CASCADE,
    FOREIGN KEY (exercice_id)
        REFERENCES exercices(id)
        ON DELETE CASCADE
    );

-- Index de performance
CREATE INDEX idx_seances_date
    ON seances(date);
CREATE INDEX idx_series_seance
    ON series(seance_id);
CREATE INDEX idx_series_exercice
    ON series(exercice_id);
CREATE INDEX idx_pe_programme
    ON programme_exercices(programme_id);
```

J. Code source complet des modèles

J.1 Modèle Programme

```
class Programme {
    final int? id;
    final String nom;
    final String description;
    final String dateCreation;

    const Programme({
        this.id,
        required this.nom,
        required this.description,
        required this.dateCreation,
    });

    Map<String, dynamic> toMap() {
        final map = <String, dynamic>{
            'nom': nom,
            'description': description,
            'date_creation': dateCreation,
        };
        if (id != null) map['id'] = id;
        return map;
    }

    factory Programme.fromMap(Map<String, dynamic> map) {
        return Programme(
            id: map['id'] as int?,
            nom: map['nom'] as String,
            description: map['description'] as String,
            dateCreation: map['date_creation'] as String,
        );
    }

    Programme copyWith({int? id, String? nom,
        String? description, String? dateCreation}) {
        return Programme(
            id: id ?? this.id,
            nom: nom ?? this.nom,
            description: description ?? this.description,
            dateCreation: dateCreation ?? this.dateCreation,
        );
    }
}
```

J.2 Modèle ProgrammeExercice

```
class ProgrammeExercice {
    final int? id;
    final int programmeId;
    final int exerciceId;
    final int jour;           // 1 = Jour 1 du programme
    final int ordre;          // Position dans le jour
    final int seriesCibles;    // Prescription : nb series
    final int repetitionsCibles; // Prescription : nb reps

    const ProgrammeExercice({
        this.id,
        required this.programmeId,
        required this.exerciceId,
        required this.jour,
        required this.ordre,
        this.seriesCibles = 3,
        this.repetitionsCibles = 10,
    });

    Map<String, dynamic> toMap() {
        final map = <String, dynamic>{
            'programme_id': programmeId,
            'exercice_id': exerciceId,
            'jour': jour,
            'ordre': ordre,
            'series_cibles': seriesCibles,
            'repetitions_cibles': repetitionsCibles,
        };
        if (id != null) map['id'] = id;
        return map;
    }

    factory ProgrammeExercice.fromMap(
        Map<String, dynamic> map) {
        return ProgrammeExercice(
            id: map['id'] as int?,
            programmeId: map['programme_id'] as int,
            exerciceId: map['exercice_id'] as int,
            jour: map['jour'] as int,
            ordre: map['ordre'] as int,
            seriesCibles: map['series_cibles'] as int,
            repetitionsCibles:
                map['repetitions_cibles'] as int,
        );
    }
}
```

J.3 Modèle Seance

```
class Seance {
    final int? id;
    final int? programmeId; // nullable : seances libres
    final String date;      // ISO 8601
    final int dureeMinutes; // Calcule automatiquement
    final String notes;     // Notes libres de l'utilisateur

    const Seance({
        this.id,
        this.programmeId,
        required this.date,
        this.dureeMinutes = 0,
        this.notes = '',
    });

    Map<String, dynamic> toMap() {
        final map = <String, dynamic>{
            'programme_id': programmeId,
            'date': date,
            'duree_minutes': dureeMinutes,
            'notes': notes,
        };
        if (id != null) map['id'] = id;
        return map;
    }

    factory Seance.fromMap(Map<String, dynamic> map) {
        return Seance(
            id: map['id'] as int?,
            programmeId: map['programme_id'] as int?,
            date: map['date'] as String,
            dureeMinutes:
                map['duree_minutes'] as int? ?? 0,
            notes: map['notes'] as String? ?? '',
        );
    }

    Seance copyWith({int? id, int? programmeId,
        String? date, int? dureeMinutes,
        String? notes}) {
        return Seance(
            id: id ?? this.id,
            programmeId: programmeId ?? this.programmeId,
            date: date ?? this.date,
            dureeMinutes: dureeMinutes ?? this.dureeMinutes,
            notes: notes ?? this.notes,
        );
    }
}
```

K. Fichier pubspec.yaml commenté

Le fichier pubspec.yaml définit les métadonnées du projet et ses dépendances. Chaque dépendance a été choisie pour une raison précise :

```
name: riki
description: Application mobile de suivi de musculation
publish_to: 'none' # Pas de publication sur pub.dev
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0' # JavaScript 3 requis (records, patterns)
  react-native: '>=3.10.0' # React Native 3.10+ pour MD3 complet

dependencies:
  react-native:
    sdk: react-native

  # Base de données SQLite embarquée.
  # react-native-sqlite-storage utilise un isolate séparé pour les I/O,
  # ne bloquant pas le thread UI.
  react-native-sqlite-storage: ^2.3.0

  # Construction de chemins de fichiers
  # (cross-platform path separator).
  path: ^1.8.3

  # Gestion d'état réactive.
  # useReducer + Consumer + MultiReact Context.
  provider: ^6.1.1

  # Graphiques interactifs (LineChart, BarChart).
  # Plus esthétique et mieux maintenu que charts_react-native.
  react-native-svg: ^0.66.0

  # Formatage des dates en français.
  # DateFormat('EEEE d MMMM yyyy', 'fr_FR').
  intl: ^0.19.0

  cupertino_icons: ^1.0.6

dev_dependencies:
  react-native_test: # Tests unitaires et widgets
    sdk: react-native
  react-native_lints: ^3.0.1 # Règles de linting JavaScript
  integration_test: # Tests d'intégration E2E
    sdk: react-native

react-native:
  uses-material-design: true # Active les icônes Material
  assets:
    - assets/ # Répertoire pour les assets futurs
```

L. Glossaire

Terme	Définition
1RM	One-Rep Max : charge maximale soulevable pour une seule répétition. Estimé par la formule d'Ep
AOT	Ahead-of-Time : compilation du code JavaScript en code machine natif avant l'exécution.
APK	Android Package Kit : format de fichier d'installation pour Android.
B-tree	Structure de données arborescente utilisée par SQLite pour les index. Recherche en $O(\log n)$.
CRUD	Create, Read, Update, Delete : les 4 opérations de base sur les données.
useReducer	Classe React Native permettant de notifier les widgets écouteurs lors d'un changement d'état.
Consumer	Widget React Native qui écoute un useReducer et se reconstruit quand notifyListeners() est appelé.
DAO	Data Access Object : couche d'abstraction entre la logique métier et la base de données.
FK	Foreign Key (clé étrangère) : référence à la clé primaire d'une autre table.
fsync	Appel système forçant l'écriture des données sur le disque. Coûteux (~10ms).
Hot reload	Fonctionnalité React Native permettant de voir les changements de code en < 1 seconde.
JIT	Just-in-Time : compilation du code JavaScript pendant l'exécution (mode développement).
MD3	Material Design 3 : système de design de Google, nativement intégré à React Native.
PK	Primary Key (clé primaire) : identifiant unique d'un enregistrement dans une table.
PR	Personal Record : record personnel sur un exercice (charge maximale atteinte).
RGPD	Règlement Général sur la Protection des Données (règlement européen).
Singleton	Design pattern garantissant qu'une seule instance d'une classe existe.
SRP	Single Responsibility Principle : chaque classe/module a une seule responsabilité.
Volume	Métrique de musculation = somme(poids x répétitions) sur une période.
WCAG	Web Content Accessibility Guidelines : normes d'accessibilité (ratio de contraste).
Widget	Composant d'interface React Native. Tout est widget dans React Native (texte, boutons, layout).

Tableau 23 — Glossaire des termes techniques

N. Code source complet des méthodes CRUD

Cette annexe présente l'intégralité des méthodes d'accès aux données du DatabaseHelper, organisées par entité.

N.1 CRUD Exercises

```
/// Récupère tous les exercices, triés par groupe puis par nom.
/// L'ORDER BY est fait coté SQLite (plus rapide qu'un sort JavaScript).
Future<List<Exercice>> getExercices() async {
  final db = await database;
  final maps = await db.query('exercices',
    orderBy: 'groupe_musculaire, nom');
  return maps.map((m) => Exercice.fromMap(m)).toList();
}

/// Filtre par groupe musculaire.
/// Utilise un paramètre positionnel (?) plutôt que la
/// concaténation de string pour éviter les injections SQL.
Future<List<Exercice>> getExercicesParGroupe(
  String groupe) async {
  final db = await database;
  final maps = await db.query('exercices',
    where: 'groupe_musculaire = ?',
    whereArgs: [groupe],
    orderBy: 'nom');
  return maps.map((m) => Exercice.fromMap(m)).toList();
}

/// Récupère un exercice par son ID.
/// Retourne null si l'ID n'existe pas.
Future<Exercice?> getExercice(int id) async {
  final db = await database;
  final maps = await db.query('exercices',
    where: 'id = ?', whereArgs: [id]);
  if (maps.isEmpty) return null;
  return Exercice.fromMap(maps.first);
}

/// Liste les groupes musculaires distincts.
/// Utilisé pour générer les FilterChips dynamiquement.
Future<List<String>> getGroupesMusculaires() async {
  final db = await database;
  final result = await db.rawQuery(
    'SELECT DISTINCT groupe_musculaire '
    'FROM exercices ORDER BY groupe_musculaire');
  return result.map(
    (m) => m['groupe_musculaire'] as String).toList();
}

/// Insère un nouvel exercice. Retourne l'ID généré.
Future<int> insertExercice(Exercice exercice) async {
  final db = await database;
  return db.insert('exercices', exercice.toMap());
}

/// Met à jour un exercice existant.
Future<int> updateExercice(Exercice exercice) async {
  final db = await database;
  return db.update('exercices', exercice.toMap(),
    where: 'id = ?', whereArgs: [exercice.id]);
}

/// Supprime un exercice par son ID.
/// Les series référençant cet exercice sont supprimées
/// automatiquement par ON DELETE CASCADE.
Future<int> deleteExercice(int id) async {
  final db = await database;
  return db.delete('exercices',
    where: 'id = ?', whereArgs: [id]);
}
```

```
/// Recherche textuelle sur nom et groupe.
/// LIKE avec % permet la recherche partielle.
Future<List<Exercice>> rechercherExercices(
    String query) async {
    final db = await database;
    final maps = await db.query('exercices',
        where: 'nom LIKE ? OR groupe_musculaire LIKE ?',
        whereArgs: ['%$query%', '%$query%'],
        orderBy: 'nom');
    return maps.map((m) => Exercice.fromMap(m)).toList();
}
```

N.2 CRUD Séances

```
/// Récupère les seances, les plus récentes d'abord.
/// Le paramètre limit évite de charger tout l'historique.
Future<List<Seance>> getSeances({int? limit}) async {
  final db = await database;
  final maps = await db.query('seances',
    orderBy: 'date DESC', limit: limit);
  return maps.map((m) => Seance.fromMap(m)).toList();
}

/// Seances d'un programme spécifique.
Future<List<Seance>> getSeancesParProgramme(
  int programmeId) async {
  final db = await database;
  final maps = await db.query('seances',
    where: 'programme_id = ?',
    whereArgs: [programmeId],
    orderBy: 'date DESC');
  return maps.map((m) => Seance.fromMap(m)).toList();
}

/// Seances entre deux dates (pour les stats).
/// Les dates sont comparées comme strings ISO 8601,
/// ce qui fonctionne car le format est lexicographiquement
/// ordonné (YYYY-MM-DD...).
Future<List<Seance>> getSeancesEntreDates(
  String debut, String fin) async {
  final db = await database;
  final maps = await db.query('seances',
    where: 'date >= ? AND date <= ?',
    whereArgs: [debut, fin],
    orderBy: 'date DESC');
  return maps.map((m) => Seance.fromMap(m)).toList();
}

/// Insertion d'une seance. Retourne l'ID généré.
/// Appelée par SeanceReact Context.sauvegarderSeance().
Future<int> insertSeance(Seance seance) async {
  final db = await database;
  return db.insert('seances', seance.toMap());
}

/// Mise à jour (durée, notes).
Future<int> updateSeance(Seance seance) async {
  final db = await database;
  return db.update('seances', seance.toMap(),
    where: 'id = ?', whereArgs: [seance.id]);
}

/// Suppression d'une seance.
/// CASCADE supprime les series associées.
Future<int> deleteSeance(int id) async {
  final db = await database;
  return db.delete('seances',
    where: 'id = ?', whereArgs: [id]);
}
```

N.3 CRUD Séries

```
/// Séries d'une seance, triées par ordre.
Future<List<Serie>> getSeriesDeLaSeance(
    int seanceId) async {
    final db = await database;
    final maps = await db.query('series',
        where: 'seance_id = ?',
        whereArgs: [seanceId],
        orderBy: 'ordre');
    return maps.map((m) => Serie.fromMap(m)).toList();
}

/// Toutes les séries d'un exercice, chronologiquement.
/// Jointure avec seances pour récupérer la date.
/// Utilisé par l'écran de détail d'un exercice.
Future<List<Serie>> getSeriesParExercice(
    int exerciceId) async {
    final db = await database;
    final maps = await db.rawQuery('''
        SELECT s.* FROM series s
        INNER JOIN seances se ON s.seance_id = se.id
        WHERE s.exercice_id = ?
        ORDER BY se.date ASC, s.ordre ASC
    ''', [exerciceId]);
    return maps.map((m) => Serie.fromMap(m)).toList();
}

/// Insertion d'une série.
Future<int> insertSerie(Serie serie) async {
    final db = await database;
    return db.insert('series', serie.toMap());
}

/// Mise à jour d'une série.
Future<int> updateSerie(Serie serie) async {
    final db = await database;
    return db.update('series', serie.toMap(),
        where: 'id = ?', whereArgs: [serie.id]);
}

/// Suppression d'une série.
Future<int> deleteSerie(int id) async {
    final db = await database;
    return db.delete('series',
        where: 'id = ?', whereArgs: [id]);
}
```


N.4 Requêtes statistiques

```
/// Record personnel (charge max) pour un exercice.
/// MAX() est une fonction d'agrégation native SQLite,
/// beaucoup plus rapide qu'un chargement complet
/// suivi d'un max() côté JavaScript.
Future<double?> getRecordPersonnel(int exerciceId) async {
  final db = await database;
  final result = await db.rawQuery(
    'SELECT MAX(poids_kg) as max_poids '
    'FROM series WHERE exercice_id = ?',
    [exerciceId]);
  if (result.isEmpty ||
    result.first['max_poids'] == null)
    return null;
  return (result.first['max_poids'] as num).toDouble();
}

/// Volume total sur une période donnée.
/// COALESCE évite un null si aucune série n'existe.
Future<double> getVolumePeriode(
  String debut, String fin) async {
  final db = await database;
  final result = await db.rawQuery('''
    SELECT COALESCE(
      SUM(s.poids_kg * s.repetitions), 0) as volume
    FROM series s
    INNER JOIN seances se ON s.seance_id = se.id
    WHERE se.date >= ? AND se.date <= ?
  ''', [debut, fin]);
  return (result.first['volume'] as num).toDouble();
}

/// Nombre de séances sur une période.
Future<int> getNombreSeancesPeriode(
  String debut, String fin) async {
  final db = await database;
  final result = await db.rawQuery(
    'SELECT COUNT(*) as count FROM seances '
    'WHERE date >= ? AND date <= ?',
    [debut, fin]);
  return (result.first['count'] as int);
}

/// Evolution des charges max par séance.
/// GROUP BY se.date + MAX(s.poids_kg) donne la charge
/// max de chaque jour d'entraînement.
Future<List<Map<String, dynamic>>> getEvolutionCharges(
  int exerciceId) async {
  final db = await database;
  return db.rawQuery('''
    SELECT se.date, MAX(s.poids_kg) as max_poids
    FROM series s
    INNER JOIN seances se ON s.seance_id = se.id
    WHERE s.exercice_id = ?
    GROUP BY se.date
    ORDER BY se.date ASC
  ''', [exerciceId]);
}

/// Volume par groupe musculaire sur une période.
/// Triple jointure : series -> exercices -> seances.
/// Les 3 index (seance, exercice, date) accélèrent
/// cette requête significativement.
Future<List<Map<String, dynamic>>> getVolumeParGroupe(
  String debut, String fin) async {
  final db = await database;
```

```

return db.rawQuery('''
    SELECT e.groupe_musculaire,
           SUM(s.poids_kg * s.repetitions) as volume
    FROM series s
    INNER JOIN exercices e ON s.exercice_id = e.id
    INNER JOIN seances se ON s.seance_id = se.id
    WHERE se.date >= ? AND se.date <= ?
    GROUP BY e.groupe_musculaire
    ORDER BY volume DESC
    ''', [debut, fin]);
}

/// Fréquence hebdomadaire (4 dernières semaines).
/// strftime('%W', date) extrait le numéro de semaine.
Future<List<Map<String, dynamic>>> getFrequenceHebdo()
    async {
    final db = await database;
    return db.rawQuery('''
        SELECT strftime('%W', date) as semaine,
               strftime('%Y', date) as annee,
               COUNT(*) as nb_seances
        FROM seances
        WHERE date >= date('now', '-28 days')
        GROUP BY annee, semaine
        ORDER BY annee, semaine
        ''');
    }
}

```

O. Code source du widget StatCard

Le StatCard est un composant « dumb » (sans logique métier) réutilisable sur le dashboard. Il reçoit ses données en props et se contente de les afficher :

```
/// Widget réutilisable pour afficher une métrique.
class StatCard extends StatelessWidget {
  final String titre;
  final String valeur;
  final String sousTitre;
  final IconData icone;
  final bool pleineLargeur;

  const StatCard({
    super.key,
    required this.titre,
    required this.valeur,
    required this.sousTitre,
    required this.icone,
    this.pleineLargeur = false,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // En-tête : icone + titre
            Row(
              children: [
                Icon(icone,
                  color: AppTheme.primaryGreen,
                  size: 20),
                const SizedBox(width: 8),
                Text(titre, style: const TextStyle(
                  color: AppTheme.textSecondary,
                  fontSize: 13)),
              ],
            ),
            const SizedBox(height: 8),

            // Valeur principale (grande police)
            Text(valeur, style: const TextStyle(
              fontSize: 28,
              fontWeight: FontWeight.w700,
              color: AppTheme.textPrimary)),
            const SizedBox(height: 2),

            // Sous-titre descriptif
            Text(sousTitre, style: const TextStyle(
              color: AppTheme.textSecondary,
              fontSize: 12)),
          ],
        ),
      ),
    );
  }
}
```

P. Analyse détaillée des formules de 1RM

Le record personnel estimé (1RM) est une métrique fondamentale en musculation. Plusieurs formules empiriques existent, chacune avec ses forces et faiblesses. Nous avons implémenté la formule d'Epley après une analyse comparative.

P.1 Les formules principales

Epley (1985) : $1RM = \text{poids} \times (1 + \text{répétitions} / 30)$

Brzycki (1993) : $1RM = \text{poids} \times 36 / (37 - \text{répétitions})$

Lander (1985) : $1RM = 100 \times \text{poids} / (101.3 - 2.67123 \times \text{répétitions})$

Lombardi (1989) : $1RM = \text{poids} \times \text{répétitions}^{0.10}$

P.2 Comparaison numérique

Le tableau suivant compare les estimations de 1RM pour une charge de 100 kg à différents nombres de répétitions :

Réps	Epley	Brzycki	Lander	Lombardi
1	100.0	100.0	101.4	100.0
2	106.7	102.9	104.2	107.2
3	110.0	105.9	107.2	111.6
5	116.7	112.5	113.7	117.5
8	126.7	124.1	125.1	123.1
10	133.3	133.3	134.1	125.9
12	140.0	144.0	144.4	128.2
15	150.0	163.6	163.3	131.1
20	166.7	211.8	208.9	134.9

Tableau 24 — Estimations de 1RM pour 100 kg (en kg)

P.3 Justification du choix d'Epley

L'analyse du tableau révèle que pour les séries de 1 à 5 répétitions, les quatre formules convergent (différence inférieure à 3%). C'est dans la plage 8-20 répétitions que les différences deviennent significatives. Brzycki tend à surestimer le 1RM pour les séries longues (divergence vers l'infini quand reps approche 37). Epley reste linéaire et plus conservatrice, ce qui est préférable pour éviter de donner à l'utilisateur une fausse impression de force. La majorité des séries d'un pratiquant intermédiaire se situent dans la plage 8-12 répétitions, où Epley offre la meilleure précision d'après les études de validation.

De plus, la formule d'Epley a l'avantage de la simplicité d'implémentation : une seule multiplication et une addition, sans risque de division par zéro (contrairement à Brzycki où reps = 37 provoque

une division par zéro). Cette robustesse est importante dans un contexte embarqué où les exceptions non gérées peuvent crasher l'application.

Q. Guide de déploiement détaillé

Q.1 Préparation de l'environnement

Le déploiement de RIKI nécessite la configuration complète de l'environnement React Native. Voici les étapes détaillées :

Étape 1 — Installation de React Native SDK : Télécharger React Native depuis `react-native.dev`. Extraire l'archive dans un répertoire (ex: `~/react-native`). Ajouter `~/react-native/bin` au PATH du système. Vérifier avec `react-native --version`.

Étape 2 — Installation d'Android Studio : Nécessaire pour le SDK Android et l'émulateur. Installer depuis `developer.android.com`. Ouvrir Android Studio, accéder à SDK Manager, installer Android SDK API 26+ (Android 8.0) et les Build Tools.

Étape 3 — Vérification : Exécuter `react-native doctor` pour vérifier que tous les composants sont installés et configurés correctement. La sortie doit afficher des coches vertes pour React Native, Android toolchain, et Android Studio.

Q.2 Compilation

Mode debug : `react-native run` compile en JIT (Just-in-Time) pour un lancement rapide avec hot reload actif. L'APK debug est plus volumineux (~50 Mo) car il embarque le runtime JIT et les symboles de débogage.

Mode release : `react-native build apk --release` compile en AOT (Ahead-of-Time) produisant du code machine natif ARM. L'APK est significativement plus petit (~15 Mo) et plus rapide au démarrage. Le tree shaking élimine automatiquement le code mort (classes et fonctions non utilisées).

Mode profile : `react-native run --profile` est un mode intermédiaire utile pour l'analyse de performance. Il compile en AOT comme le release mais inclut les outils de profiling (timeline, memory inspector).

Q.3 Distribution

Pour distribuer l'APK sans passer par le Play Store, trois méthodes sont possibles : (1) transfert direct du fichier .apk par USB, email ou lien de téléchargement, (2) installation via ADB (`adb install app-release.apk`), (3) utilisation d'un service de distribution comme Firebase App Distribution pour faciliter les tests internes.

Pour une distribution via le Google Play Store (hors périmètre de ce projet), il faudrait générer un App Bundle (`react-native build appbundle`) plutôt qu'un APK, signer l'application avec une clé de publication, créer une fiche Play Store avec captures d'écran et description, et soumettre le bundle pour revue.

R. Concepts de musculation appliqués

Cette annexe présente les concepts fondamentaux de musculation sur lesquels repose la logique métier de RIKI.

R.1 Le volume d'entraînement

Le volume d'entraînement est défini comme la somme des charges multipliées par les répétitions sur une période donnée. Par exemple, si un pratiquant effectue 3 séries de développé couché à 80 kg x 10 reps, le volume pour cet exercice est de $3 \times 80 \times 10 = 2\,400$ kg. La littérature scientifique montre que le volume est le déterminant principal de l'hypertrophie musculaire (croissance du muscle), davantage que l'intensité (charge relative au 1RM) ou la fréquence seule.

Dans RIKI, le volume est calculé à deux niveaux : par série (getter volume = poidsKg x repetitions dans le modèle Serie) et par période (requête SQL SUM dans getVolumePeriode). Le tableau de bord affiche le volume hebdomadaire, permettant à l'utilisateur de vérifier qu'il maintient une charge de travail suffisante.

R.2 La surcharge progressive

Le principe de surcharge progressive stipule que pour progresser, le pratiquant doit augmenter graduellement le stress imposé au muscle. Cela peut se faire en augmentant la charge (ajouter 2.5 kg au développé couché), le nombre de répétitions (à charge égale), ou le nombre de séries. La détection de records personnels dans RIKI est un indicateur direct de surcharge progressive réussie : si la charge max augmente sur un exercice, le pratiquant progresse.

R.3 Les splits d'entraînement

Un split désigne la manière dont les groupes musculaires sont répartis sur les jours de la semaine. Les trois splits les plus populaires, présents comme modèles dans RIKI, sont :

- **PPL (Push/Pull/Legs)** : division en mouvements de poussée (pectoraux, épaules, triceps), traction (dos, biceps) et jambes. Idéal pour 6 séances/semaine (chaque groupe travaillé 2 fois).
- **Upper/Lower** : alternance haut/bas du corps. 4 séances/semaine. Bon compromis volume/récupération pour les intermédiaires.
- **Full Body** : corps entier à chaque séance. 3 séances/semaine. Recommandé pour les débutants (fréquence élevée par groupe, volume modéré par séance).

R.4 Le temps de repos

Le temps de repos entre les séries influence directement la performance et les adaptations physiologiques. Pour l'hypertrophie, un repos de 60 à 120 secondes est généralement recommandé. Pour la force maximale, 3 à 5 minutes sont nécessaires pour la résynthèse complète de la phosphocréatine (source d'énergie des efforts courts et intenses). Le chronomètre intégré de RIKI propose trois presets (60s, 90s, 120s) couvrant la plage standard d'hypertrophie. Un repos trop court réduit la performance sur les séries suivantes ; un repos trop long allonge inutilement la séance et réduit le stress métabolique (un des mécanismes de l'hypertrophie).

M. Références

- (1) Documentation officielle React Native — <https://docs.react-native.dev>
- (2) Documentation `react-native-sqlite-storage` — <https://pub.dev/packages/react-native-sqlite-storage>
- (3) Documentation `react-native-svg` — <https://pub.dev/packages/react-native-svg>
- (4) Documentation React Context — <https://pub.dev/packages/provider>
- (5) Material Design 3 — <https://m3.material.io>
- (6) SQLite documentation officielle — <https://www.sqlite.org/docs.html>
- (7) Epley, B. (1985). Poundage Chart. Boyd Epley Workout.
- (8) Brzycki, M. (1993). Strength Testing — Predicting a One-Rep Max from Reps-to-Fatigue. JOPERD, 64(1), 88-90.
- (9) Schoenfeld, B.J. (2010). The mechanisms of muscle hypertrophy and their application to resistance training. JSCR, 24(10).
- (10) Google Developers. JavaScript language specification. <https://dart.dev/guides>
- (11) Android Developers. Room vs SQLite. <https://developer.android.com>
- (12) WCAG 2.1 — Web Content Accessibility Guidelines. <https://www.w3.org/WAI/standards-guidelines/wcag/>